

数据结构

Ch10 内部排序

北京邮电大学 计算机学院

第10章 内部排序

- 10.1 引言
- 10.2 插入排序
 - 10.2.1 直接插入排序
 - 10.2.2 希尔排序
- 10.3 交换排序
 - 10.3.1 冒泡排序
 - 10.3.2 快速排序
- 10.4 选择排序
 - 10.4.1 简单选择排序
 - 10.4.2 树形选择排序
 - 10.4.3 堆排序
- 10.5 归并排序
- 10.6 分配排序
 - 10.6.1 桶排序
 - 10.6.2 计数排序
 - 10.6.3 基数排序
- 10.7 排序方法比较

什么是排序

其目的是将一组“**无序**”的记录序列调整为“**有序**”的记录序列，即根据记录关键字（排序码）的值的递增(递减)的关系将记录（数据元素）的次序重新排列。

[例] 将下列关键字序列：

52, 49, 80, 36, 14, 58, 61, 23, 97, 75

调整为：

14, 23, 36, 49, 52, 58, 61, 75, 80, 97

非递减序列、递减序列、非递增序列、递增有序

$\leq$

$>$

$\geq$

$<$

<https://github.com/hustcc/JS-Sorting-Algorithm>

<https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>

排序的分类

[根据排序时文件记录的存放位置]

- **内部排序**: 排序过程中将全部记录放在内存中处理。
- **外部排序**: 排序过程中需在内外存之间交换信息。

[根据排序前后相同关键字记录的相对次序]

- **稳定排序**: 设序列中任意两个记录的关键字值相同, 即 $K_i = K_j (i \neq j)$, 若排序之前记录 R_i 领先于记录 R_j , 排序后这种关系不变(对所有输入实例而言)。
- **不稳定排序**: 只要有一个实例使排序算法不满足稳定性要求。

[根据排序的方法]

插入排序

交换排序

选择排序

归并排序

基数排序

[根据排序算法所需的辅助空间]

- 就地排序: $O(1)$
- 非就地排序: $O(n)$ 或与 n 有关

评价排序算法的主要标准

[时间开销]

考察算法的两个基本操作的次数：

- 比较关键字
- 移动记录

算法时间还与输入实例的初始状态有关时，分情况：

- | | |
|------|----------------------|
| • 最好 | 一般来说 |
| • 最坏 | 简单排序方法： $O(n^2)$ |
| • 平均 | 先进排序方法： $O(n\log n)$ |

[空间开销]

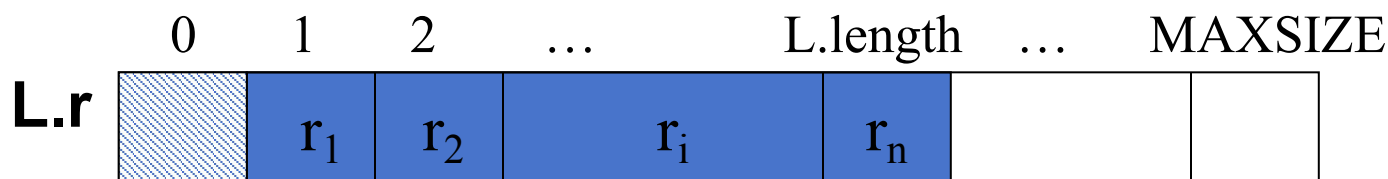
所需的辅助空间

讨论约定本章讨论中，除特殊声明外，一般采用**顺序结构存储**，
按记录关键字**非递减**排序，关键字为**整数**

```
#define MAXSIZE 20
typedef int KeyType;
typedef struct {
    KeyType key;
    InfoType otherinfo;
} RedType;
typedef struct {
    RedType r[MAXSIZE+1]; //r[0]闲置或作哨兵
    int length;
}SqList;
```

SqList L;

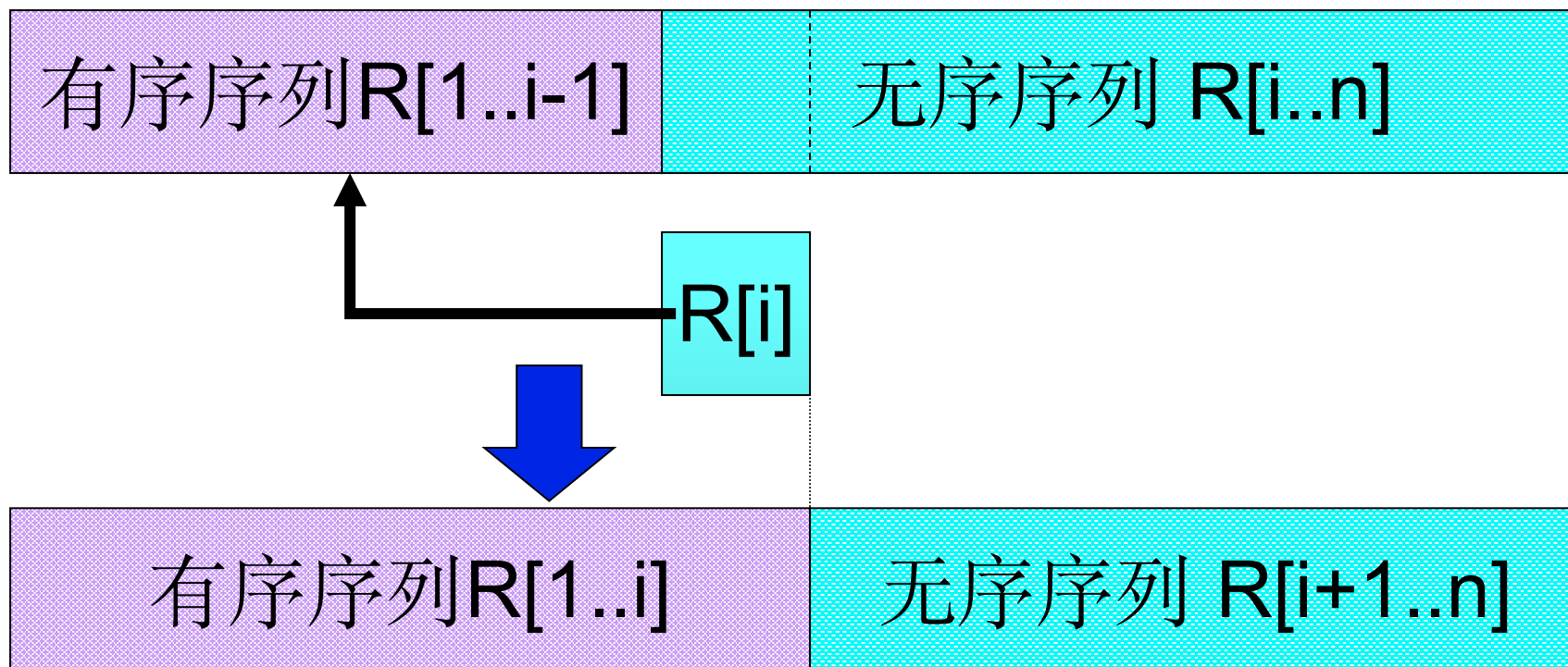
R[0]闲置或做监视哨



10.2 插入排序



插入排序的基本思想是：每次将一个待排序的记录，按其关键字大小插入到前面已经排好序的子表的适当位置，直到全部记录插入完成，整个表有序为止。



10.2.1 直接插入排序

直接插入排序是一种简单的插入排序方法，基本思想为：在 $R[1]$ 至 $R[i-1]$ 长度为 $i-1$ 的子表已经有序的情况下，查找 $R[i]$ 的插入位置后将 $R[i]$ 插入，得到 $R[1]$ 至 $R[i]$ 长度为 i 的子表有序，这样通过 $n-1$ 趟（ $i=2..n$ ）之后， $R[1]$ 至 $R[n]$ 有序。

[示例] { $R(0)$ $R(-4)$ $R(8)$ $R(1)$ $R(-4)$ $R(-6)$ } $n=6$

$i=1$ [0] -4 8 1 -4 -6

$i=2$ [-4 0] 8 1 -4 -6

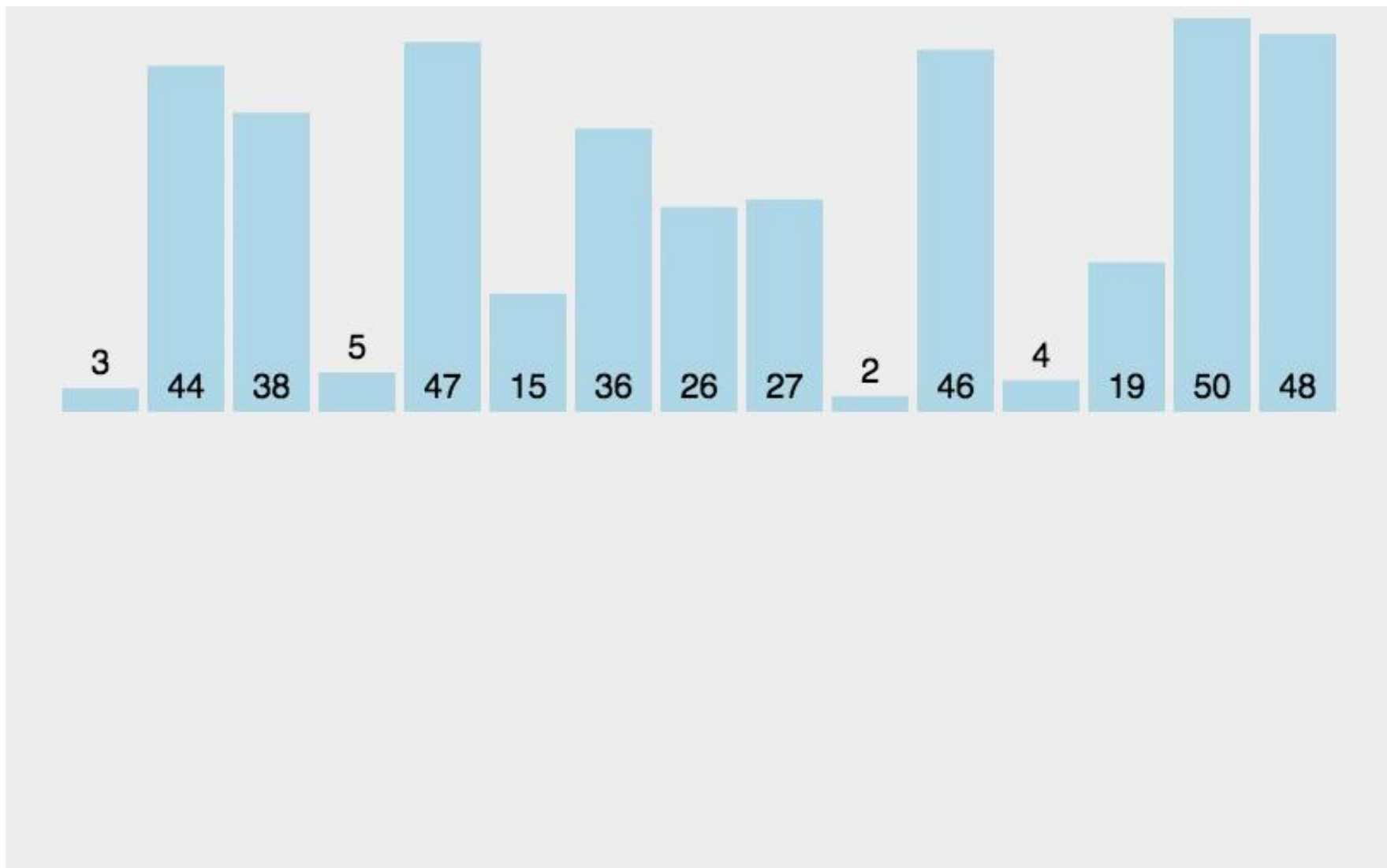
$i=3$ [-4 0 8] 1 -4 -6

$i=4$ [-4 0 1 8] -4 -6

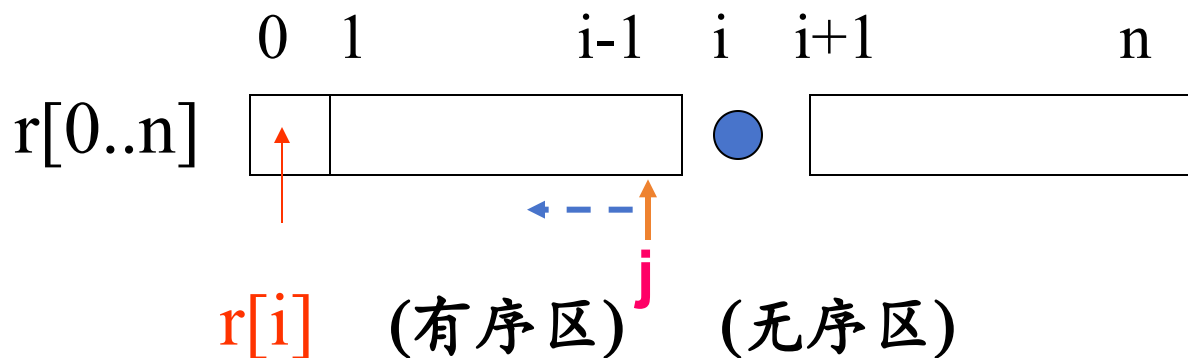
$i=5$ [-4 -4 0 1 8] -6

$i=6$ [-6 -4 -4 0 1 8]

稳定排序



[算法步骤]



作为“监视哨”

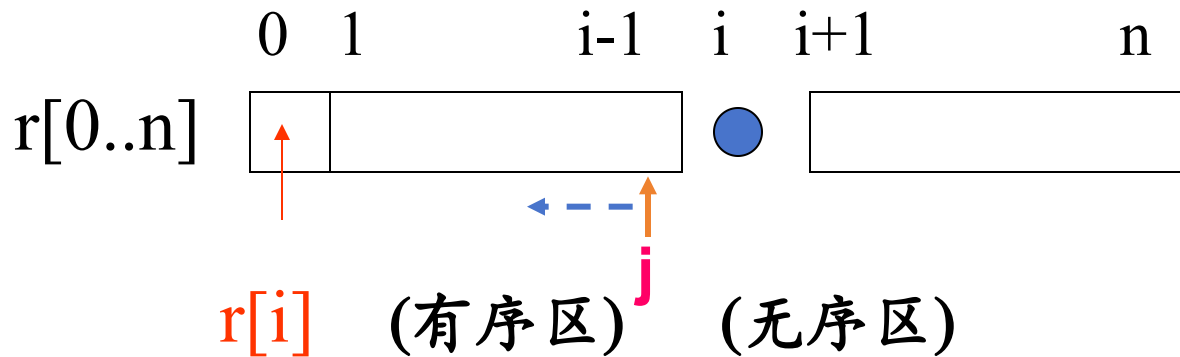
循环 $(n-1)$ 次，初值 $i=2$

1) 若 $r[i] < r[i-1]$ ，则把第 i 个记录取出保存在 $r[0]$ 中， $j=i-1$

2) 若 $r[0] < r[j]$ ，则 $r[j]$ 后移一位， $j=j-1$ ，转2)；

否则 $r[0]$ 放在 $r[j+1]$ 处， $i=i+1$ ，转1)

[算法步骤]



```
void InsertSort (SqList &L)
{ for (i=2; i<=L.length; i++) //n-1 轮
    if (LT(L.r[i].key, L.r[i-1].key)) { //比前面的小
        L.r[0] = L.r[i]; L.r[i] = L.r[i-1]; //监视哨
        for (j=i-2; L.r[0].key < L.r[j].key; j--) //逐一比较
            L.r[j+1] = L.r[j]; //向后移动一个位置
        L.r[j+1] = L.r[0]; //插入
    }
} //InsertSort
```

【性能分析】

空间性能：仅用了一个辅助单元 $R[0]$ 作为监视哨，空间复杂度为 $O(1)$ 。

时间性能：向有序表中逐个插入记录的操作，进行了 $n-1$ 趟，每趟操作分为比较关键码和移动记录，而比较的次数和移动记录的次数取决于初始序列的排列情况。分三种情况讨论：

(1) 最好情况 (**初始正序**) 下: 每趟操作只需1次比较, 0次移动

总比较次数= $n-1$ 次

总移动次数= 0次

所以时间复杂度为 $O(n)$

(2) 最坏情况 (**初始逆序**) 下: 即每一趟操作都要插入记录到最前面。

和前面的j-1个关键字比+监视哨比

$$\text{总比较次数} = \sum_{j=2}^n j = \frac{1}{2}(n+2)(n-1)$$

$$\text{总移动次数} = \sum_{j=2}^n (j+1) = \frac{1}{2}(n+4)(n-1)$$

所以时间复杂度为 $O(n^2)$

进监视哨1次
+前面的j-1个依次后移
+监视哨移动到R[1]

稳定性: 直接插入排序是一个**稳定**的排序

[改进措施]

- 折半插入排序

算法思想：将循环中每一次在区间 $[1, i-1]$ 上为确定插入位置的顺序查找操作改为折半查找操作。

效果：减少关键字间的比较次数，移动次数不变。

- 表插入排序

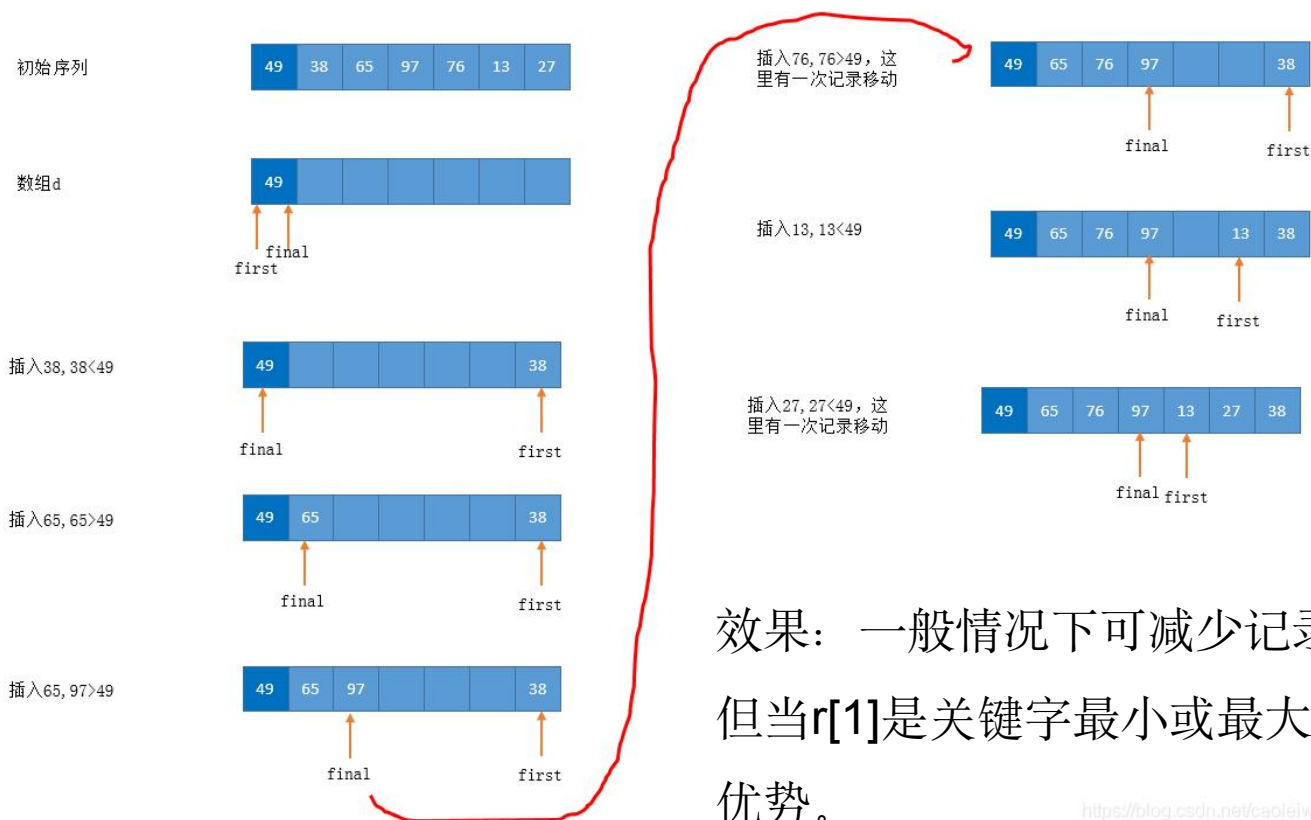
算法思想：构造链表，用改变指针来代替移动记录操作

效果：减少记录的移动次数。

[改进措施]

• 2-路插入排序

算法思想：设置与r同样大小的辅助空间d，将r[1]赋值给d[1]，将d看作循环向量。对于r[i] ($2 \leq i \leq n$)，若 $r[i] \geq d[1]$ ，则插入d[1]之后的有序序列中，反之则插入d[1]之前的有序序列中。



10.2.2 希尔排序(渐减/缩小增量排序)

直接插入排序算法简单，特点为：在 n 值较小时，效率较高，在 n 值很大时，若序列按关键码基本有序，效率依然较高，其时间效率可提高到 $O(n)$ 。

希尔排序(Shell's Sort)是从这两点出发，给出插入排序的改进方法。希尔排序又称缩小增量排序，是1959年由D.L.Shell提出来的。

核心思路：在有序性差时尽量不让排序表太大，
随着有序性变好，排序表逐渐加大。

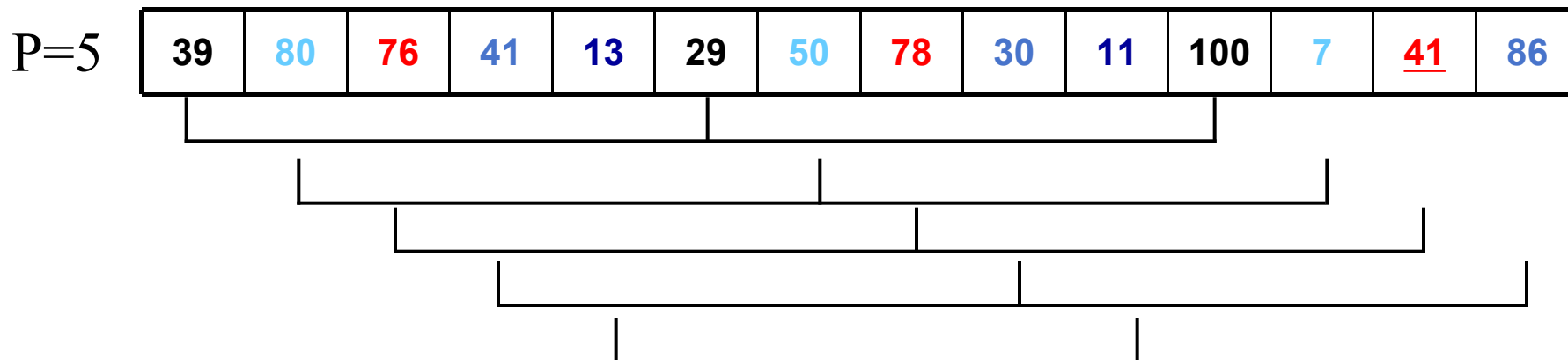
希尔排序的思想:

- (1) 先选取一个小于 n 的整数 d_i (称之为步长)
- (2) 然后把排序表中的 n 个记录分为 d_i 个组, 从第一个记录开始, 间隔为 d_i 的记录为同一组, 各组内进行直接插入排序。
- (3) 一趟之后, 间隔 d_i 的记录有序, 随着有序性的改善, 减小步长 d_i (排序子表变大), 重复进行, 直到 $d_i=1$ (全部记录成为一个排序表), 使得间隔为1的记录有序, 也就使整体达到了有序。

2、【例】排序列表为：

39, 80, 76, 41, 13, 29, 50, 78, 30, 11, 100, 7, 41, 86

步长因子分别取5、3、1，则排序过程如下：



间隔为5的子序列分别为：{39, 29, 100}, {80, 50, 7}, {76, 78, 41}, {41, 30, 86}, {13, 11}。

跑41前面了，非稳定排序

第一趟排序结果，使得间隔为5的字表有序：

P=3

29	7	<u>41</u>	30	11	39	50	76	41	13	100	80	78	86

子序列分别为： $\{29, 30, 50, 13, 78\}$ ， $\{7, 11, 76, 100, 86\}$ ，

$\{41, 39, 41, 80\}$ 。第二趟排序结果：

P=1

13	7	39	29	11	<u>41</u>	30	76	41	50	86	80	78	100
----	---	----	----	----	-----------	----	----	----	----	----	----	----	-----

此时，序列“基本有序”，对其进行直接插入排序，得到最终结果：

7	11	13	29	30	39	<u>41</u>	41	50	76	78	80	86	100
---	----	----	----	----	----	-----------	----	----	----	----	----	----	-----

10.3 交换排序

[算法思想]

将两个相邻记录的关键字进行比较，若为逆序则交换两者位置，小者往上浮，大者往下沉。

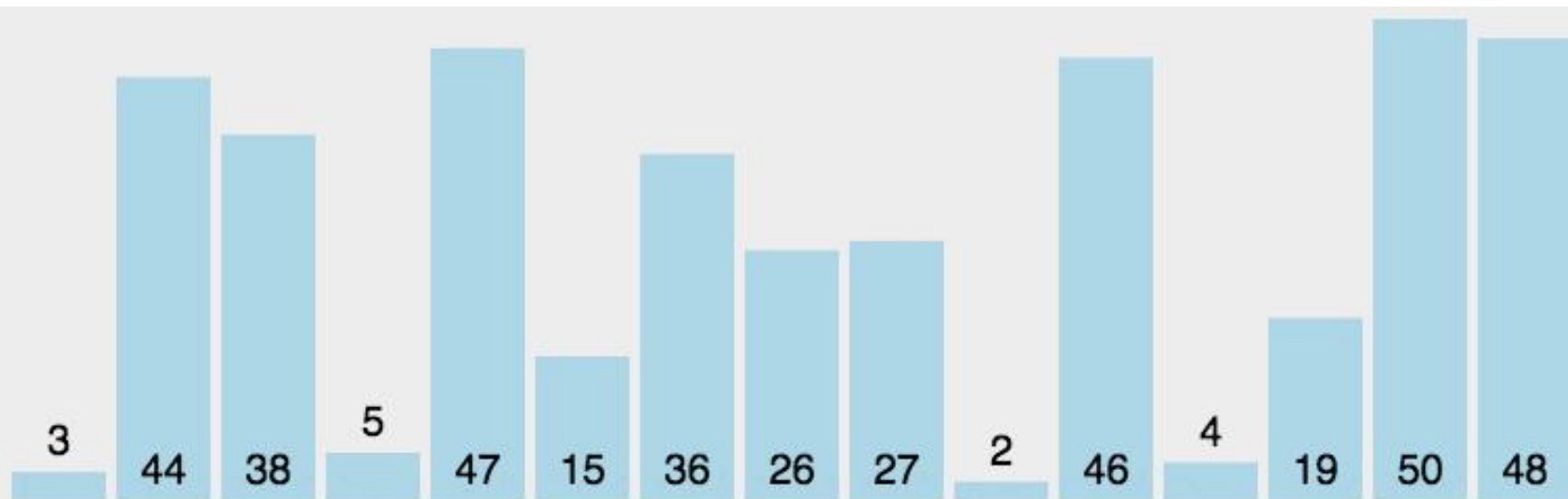
[算法步骤]

记录1和2、2和3、……、 $(n-1)$ 和 n 的关键字比较(交换);

记录1和2、2和3、……、 $(n-2)$ 和 $(n-1)$ 的关键字比较(交换);

……

直到某一趟不出现交换操作为止。



[示例]

<初态>	<第一趟>	<第二趟>	<第三趟>	<第四趟>	稳定排序
0	-4	-4	-6	-6	
-4	0	-6	-4	-4	
8	-6	<u>-4</u>	<u>-4</u>	<u>-4</u>	
-6	<u>-4</u>	0	0	0	
<u>-4</u>	1	1	1	1	
1	8	8	8	8	

sorted

F

F

F

T

[算法描述]

```
void BubbleSort (SqList &L)
{
    n=L.length; sorted=FALSE;
    for (i=0;i<n-1 && (! sorted) ;i++) //最多n-1趟
        sorted=TRUE;
        for (j=1;j<n-i;j++) //对前n-i个相邻元素逐个两两相比
            if (L.r[j].key>L.r[j+1].key) { //是否逆序
                L.r[j]←→L.r[j+1]; //交换
                sorted=FALSE; //设置标志位, 表明本趟有交换
            }
    }
}

} // BubbleSort
```


[性能分析]

- 最好情况(原始数据**正序**), 只需**一趟**, $n-1$ 次比较, 不需要移动。

$$C_{\min}=n-1 \quad M_{\min}=0$$

- 最坏情况(原始数据**逆序**), 需 $n-1$ 趟, 第 i 趟 $n-i$ 次比较, $3*(n-i)$ 次移动

$$C_{\max} = \sum_{i=1}^{n-1} (n-i) = \frac{1}{2}(n^2 - n) \quad M_{\max} = 3 \sum_{i=1}^{n-1} (n-i) = \frac{3}{2}(n^2 - n)$$

- 时间复杂度 **$O(n^2)$** , 辅助空间复杂度 **$O(1)$**

[算法的改进]

- 每趟排序中, 记录最后一次发生交换的位置

例: {54, 32, 16, 55, 103, 727, 834, 1006}

- 双向交替扫描, 改进数据不对称性:

上→下, 最重沉底; 下→上, 最轻升顶。

例: {12, 18, 42, 44, 45, 67, 94, 10}

10.3.2 快速排序

1.快速排序的思想

快速排序是通过比较关键码、交换记录，以**某个记录为界**（该记录称为支点），**将待排序列分成两部分**。一部分是关键码**大于等于支点**记录的，另一部分是关键码**小于支点**记录的关键码。

我们将待排序列按关键码以支点记录分成两部分的过程，称为**一次划分**。

对各部分不断划分，直到整个序列按关键码有序。

快速排序

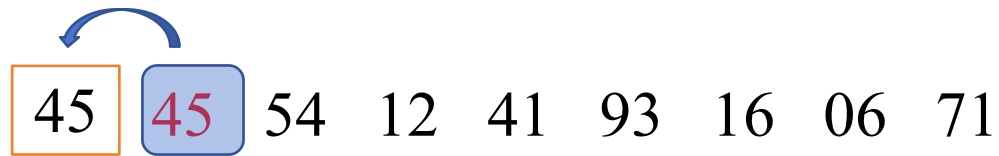


45 54 12 41 93 16 06 71



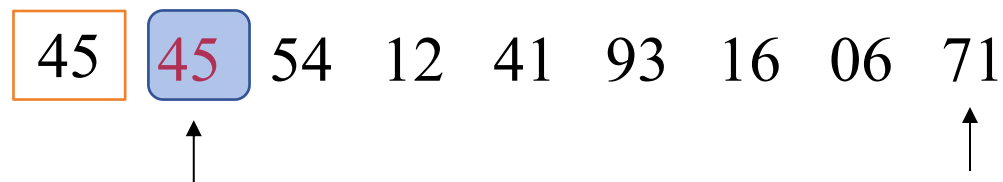
辅助空间

快速排序



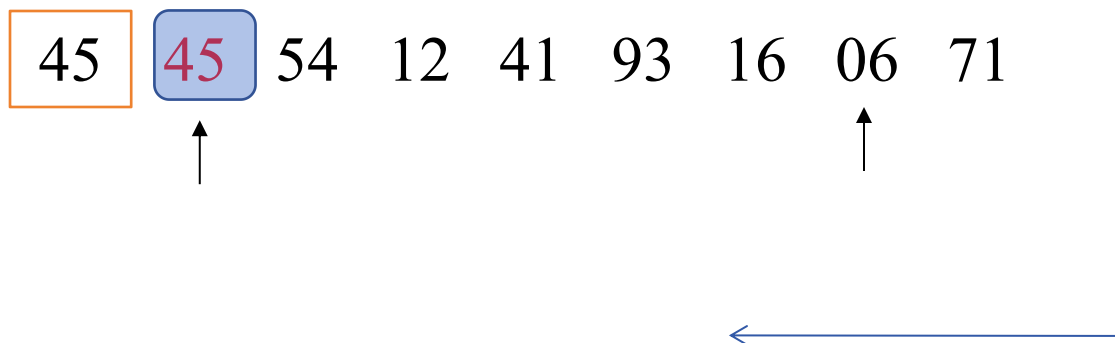
可以被覆盖的空间

快速排序



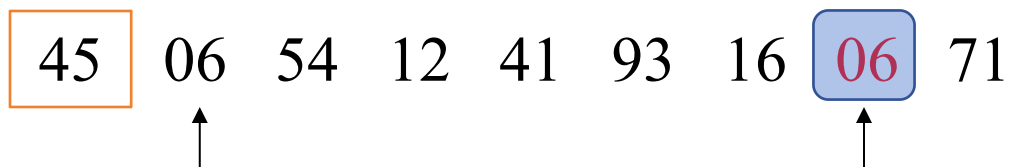
从后向前，大于等于45跳过

快速排序



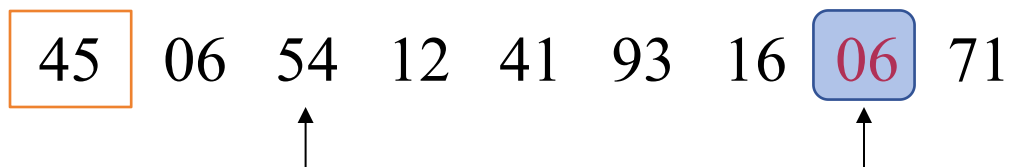
从后向前，大于等于45跳过，
小于45，则将其移动到原45所在的位置

快速排序



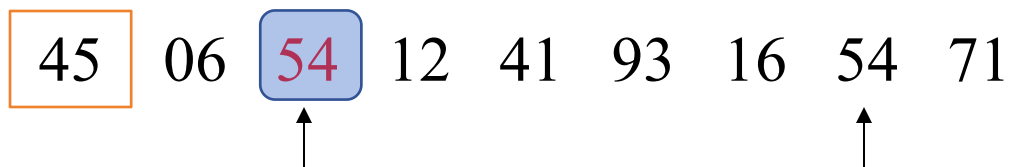
将06移动到原45所在的位置后，06原来的位置即可被使用

快速排序



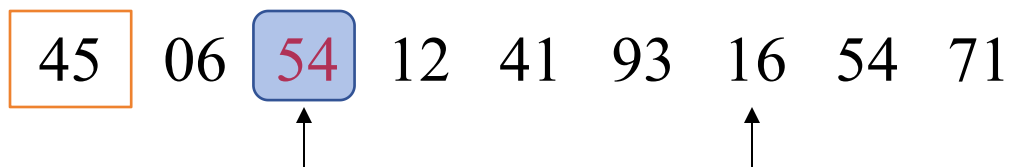
从前向后，小于等于45跳过，
大于45，则将其移动到原06所在的位置

快速排序



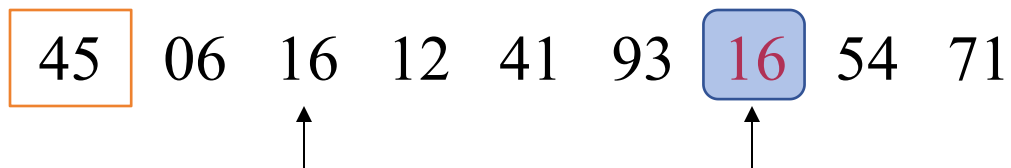
54大于45，则将54移动到原06所在的位置，
54原来的位置即可被使用

快速排序



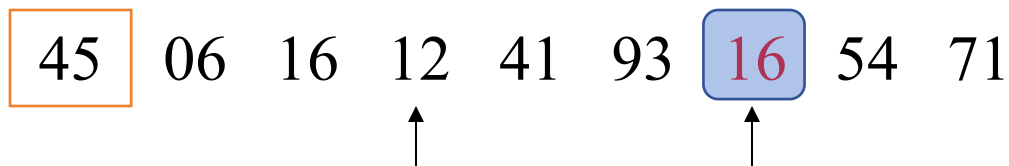
从后向前，大于等于45跳过，
小于45，则将其移动到原54所在的位置

快速排序



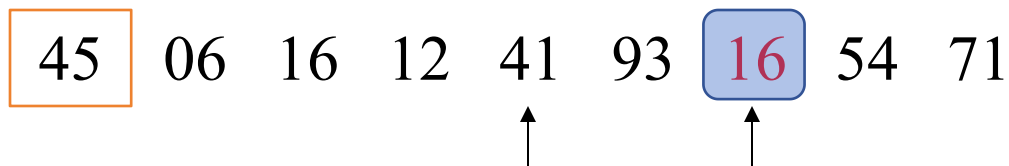
将16移动到原54所在的位置后，16原来的位置即可被使用

快速排序



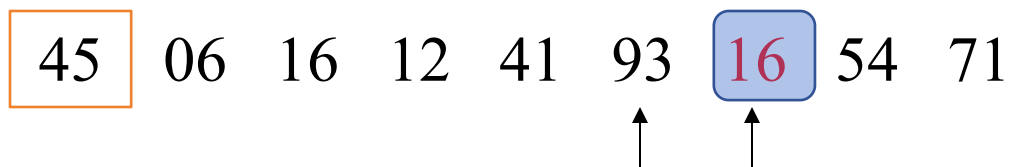
从前向后，小于等于**45**跳过，
大于**45**，则将其移动到原**16**所在的位置

快速排序



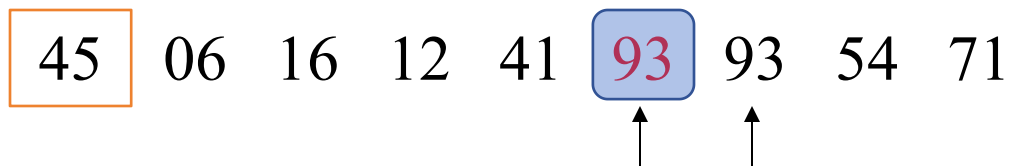
从前向后，小于等于**45**跳过，
大于**45**，则将其移动到原**16**所在的位置

快速排序



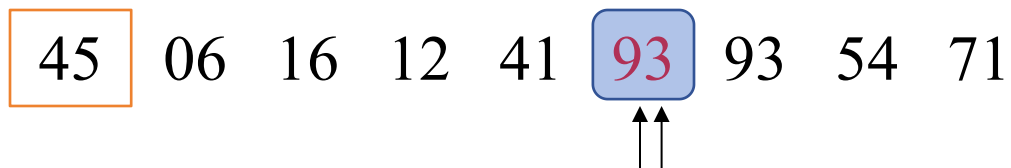
从前向后，小于等于**45**跳过，
大于**45**，则将其移动到原**16**所在的位置

快速排序



将93移动到原16所在的位置后，93原来的位置即可被使用

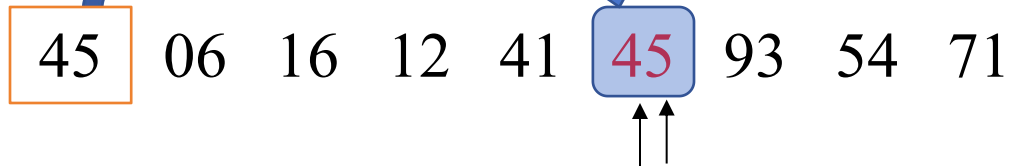
快速排序



继续从后向前, low、high相遇

快速排序

第一趟:



快速排序

第一趟: 06 16 12 41 **45** 93 54 71

The diagram illustrates the first pass of a quicksort algorithm. The array is [06, 16, 12, 41, 45, 93, 54, 71]. The pivot is 45, highlighted in red. Brackets below the array group elements less than 45 (06, 16, 12, 41) and elements greater than 45 (93, 54, 71).

一次划分后，再分别对支点前后的两组继续划分下去，直到每一组只有一个记录为止。

快速排序

第一趟: 06 16 12 41 **45** 93 54 71

第二趟: **06** 16 12 41 **45** 71 54 **93**

快速排序

第一趟: 06 16 12 41 **45** 93 54 71

第二趟: **06** 16 12 41 **45** 71 54 **93**

快速排序

第一趟: 06 16 12 41 45 93 54 71

第二趟: 06 16 12 41 45 71 54 93

第三趟: 06 12 16 41 45 54 71 93

快速排序

第一趟: 06 16 12 41 45 93 54 71

第二趟: 06 16 12 41 45 71 54 93

第三趟: 06 12 16 41 45 54 71 93

第四趟: 06 12 16 41 45 54 71 93

[算法描述]

45

45

54

12

41

93

16

06

71

int Partition (SqList &L, int low, int high) ↑

{ L.r[0] = L.r[low]; //将low记录移动到r[0]

pivotkey = L.r[low].key; //low的key作为支点

while (low < high) {

while (low < high && L.r[high].key >= pivotkey) --high;

//从后向前，如果high对应的key大于等于支点，则跳过(--high)

L.r[low] = L.r[high];

//high对应的key小于支点，将其移动到low里面

while (low < high && L.r[low].key <= pivotkey) ++low;

// 从前向后，如果low对应的key小于等于支点，则跳过(++low)

L.r[high] = L.r[low];

}

L.r[low]=L.r[0];

return low;

}//Partition



[算法描述]

```
void QuickSort ( SqList &L )  
// 对顺序表 L 进行快速排序  
{  
    QSort ( L, 1, L.length );  
} // QuickSort
```

```
void QSort ( SqList &L, int low, int high )  
//对顺序表 L 中的子序列L.r[low..high] 进行快速排序  
{  
    if ( low < high ) {  
        pivotloc = Partition(L, low, high );  
        Qsort (L, low, pivotloc-1);  
        Qsort (L, pivotloc+1, high )  
    }  
} // QSort
```


时间效率：在 n 个记录的待排序列中，一次划分需要约有 $\leq n$ 次关键码比较，时效为 $O(n)$

理想情况下：每次划分，正好将分成两个等长的子序列，则需要的排序趟数为 $\leq \log_2 n$ ，故时间性能为 $O(n \log_2 n)$ 。

最坏情况下（初始正序/逆序）：即每次划分只得到一个子序列时，时间效率为 $O(n^2)$

快速排序通常被认为是在同数量级 $O(n \log_2 n)$ 的排序方法中平均性能最好的。

快速排序是一个不稳定的排序。

反例： 3, 2, 2 $\Rightarrow$ 选择3为支点, [3] — 2, 2 $\Rightarrow$ 2, 2, 3



空间效率：快速排序是递归的，每层递归调用时的指针和参数均要用栈来存放，递归调用层次数与上述二叉树的深度一致。因而，存储开销在理想情况下为 $O(\log_2 n)$ ，即树的高度；在最坏情况下，即二叉树是一个单链，为 $O(n)$ 。

[算法的改进]

合理选择枢轴记录可改善性能。例如，

- 三者取中
- 随机产生

10.4 选择排序

10.4.1 简单选择排序(直接选择排序)

[算法思想]

每次从待排序列中选出关键字最小记录作为有序序列中最后一个记录,直至最后一个记录为止。

[示例] (n=8)

	此时49在49前					选出最小的		
<初态>	49	38	65	97	76	49	13	27
<第1趟>	13	38	65	97	76	49	49	27
<第2趟>	13	27	65	97	76	49	49	38
<第3趟>	13	27	38	97	76	49	49	65
<第4趟>	13	27	38	49	76	97	49	65
<第5趟>	13	27	38	49	49	97	76	65
<第6趟>	13	27	38	49	49	65	76	97
<第7趟>	13	27	38	49	49	65	76	97

不稳定排序

(每趟排序使有序区增加一个记录)

[算法描述]

```
void SelectSort(SqList &L)
{ for (i=1;i<L.length;i++) //n-1趟
    { j=i;
      for (k= i + 1; k<=L.length; k++)
        if (L.r [ j ] .key > L.r [ k ].key) j= k; //找到最小
      if (i!=j) L.r[i]  $\longleftrightarrow$  L.r[j];
    }
} //SelectSort
```

简单选择排序的性能分析

空间效率：仅用了**一个辅助单元**R[0]作为交换的中介。

时间效率：简单选择排序**移动**记录的次数**较少**

最好情况下移动0次

最坏情况下，每趟排序都需要交换（即每趟需移动3次），**共需移动** $3(n-1)$ 次；

关键码的**比较次数与初始序列情况无关**，第i趟需要比较 $n-i$ 次，

$$(n-1)+(n-2)+(n-3)+\cdots+1=n(n-1)/2;$$

所以算法的时间复杂度为 $O(n^2)$

简单选择排序是一个**不稳定**的排序。

简单选择排序的思想简单，易于实现，但其时间性能没有优势，这是因为在每趟的选择中，没有把前面选择过程中的一些有用信息**继承**下来，因此每趟选择都是顺序的一一进行，如果某一趟的选择能够把前面有用的一些信息继承下来，则定会减少本趟的比较次数，提高排序效率。

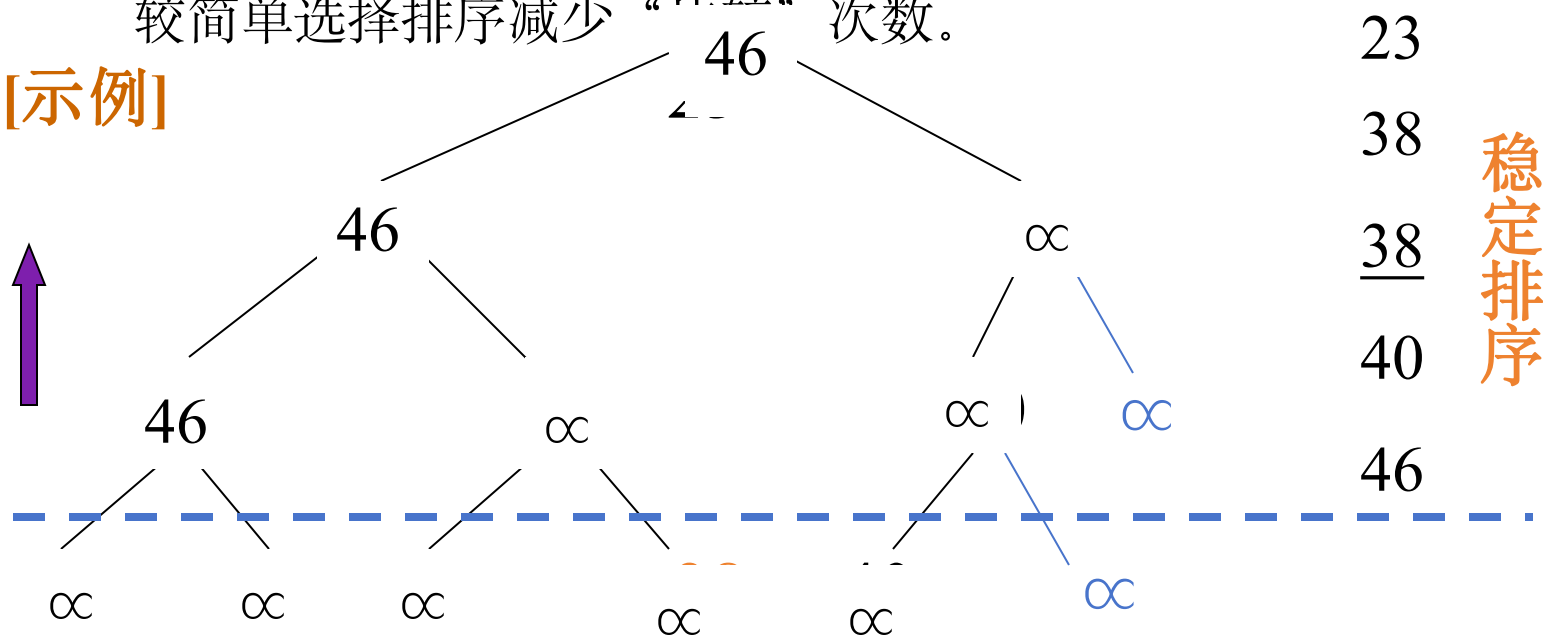
10.4.2 树形选择排序(锦标赛排序)

[算法思想]

锦标赛名次产生过程。

较简单选择排序减少“ $\frac{n(n-1)}{2}$ ”次数。

[示例]



[性能] 除第一次外，每次都走了树的一条分支，故其时间复杂度为 $O(n \log_2 n)$ 。

[缺陷] 辅助空间较多；需与“最大值”进行多余的比较

10.4.3 堆排序

1. 完全二叉堆的定义

设有 n 个元素的序列 $R_1, R_2, \dots, R_n$, 当且仅当满足下述关系之一时, 称之为完全二叉堆。

$$R_i \leq \begin{cases} R_{2i} \\ R_{2i+1} \end{cases} \quad \text{或} \quad R_i \geq \begin{cases} R_{2i} \\ R_{2i+1} \end{cases} \quad \text{其中 } i=1, 2, \dots, n/2$$

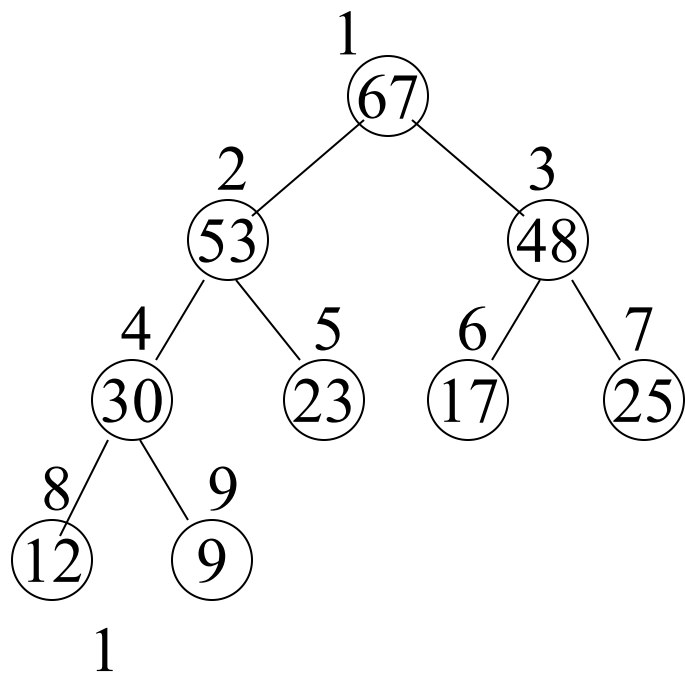
前者称为小顶堆, 后者称为大顶堆。

如: 序列 12, 36, 24, 85, 47, 30, 53, 91 是一个小顶堆;

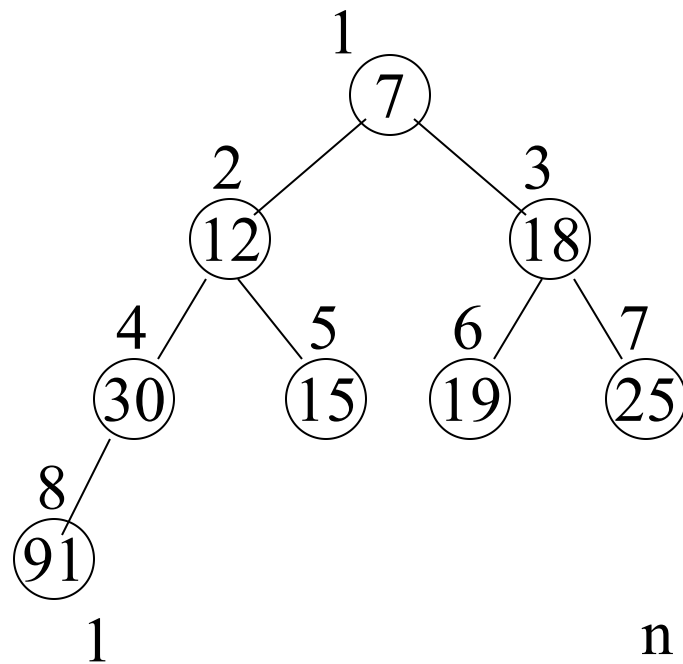
序列 91, 47, 85, 24, 36, 53, 30, 16 是一个大顶堆。

完全二叉堆

大顶堆



小顶堆



顺序存储

67	53	48	30	23	17	25	12	9
----	----	----	----	----	----	----	----	---

$$k_i \geq \begin{cases} k_{2i} \\ k_{2i+1} \end{cases}$$

7	12	18	30	15	19	25	91	
---	----	----	----	----	----	----	----	--

$$k_i \leq \begin{cases} k_{2i} \\ k_{2i+1} \end{cases}$$

2 . 堆排序

堆特点：堆顶元素是整个序列中最大(或最小)的元素。

- 首先将排序表按关键码建成堆，堆顶元素就是最大元素（或最小），将其输出
- 再对剩下的 $n-1$ 个元素建成堆，得到次大(或次小)的元素。以此类推，直到进行 $n-1$ 次后，排序结束，便得到一个按关键码有序的序列。称这个过程为堆排序。

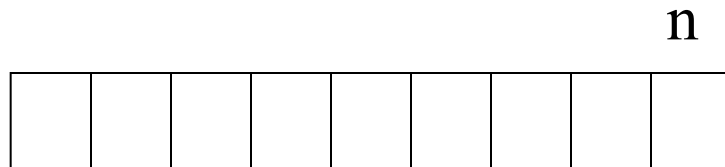
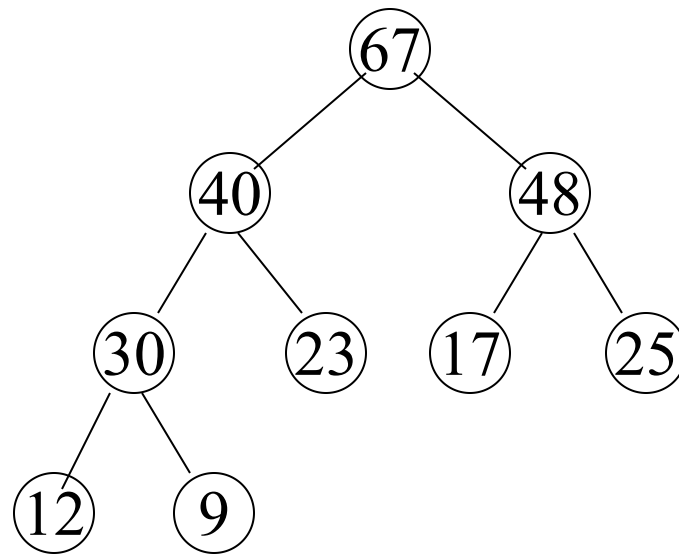
因此，实现堆排序需解决两个问题：

1. 如何将 n 个元素的排序序列按关键码建成堆（初始堆）；
2. 怎样将剩余的 $n-1$ 个元素按其关键码调整为一个新堆。

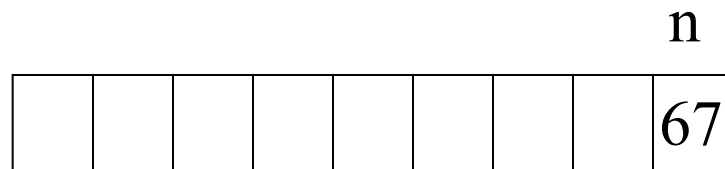
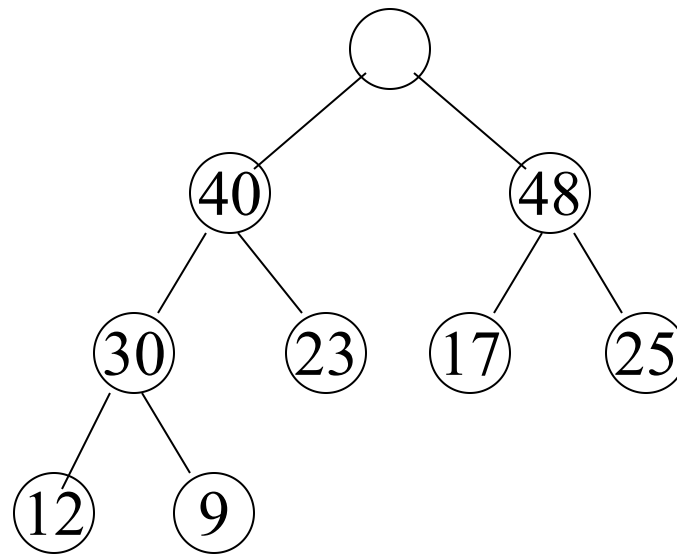
堆排序

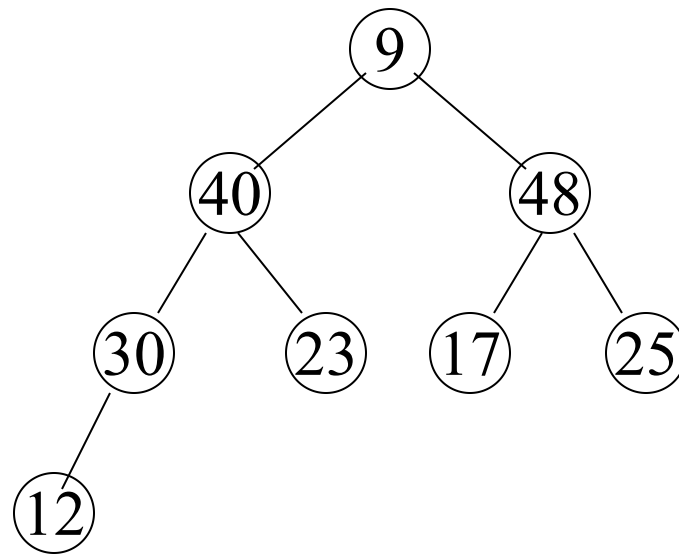
算法思想：

- 1 取出根元素，放在 $R[n]$
- 2 将剩下的 $n-1$ 个元素重新调整为堆
- 3 再取出根元素，放在 $R[n-1]$ ，将剩下的元素再调整为堆
- 4 如此反复，直到取尽堆中所有元素

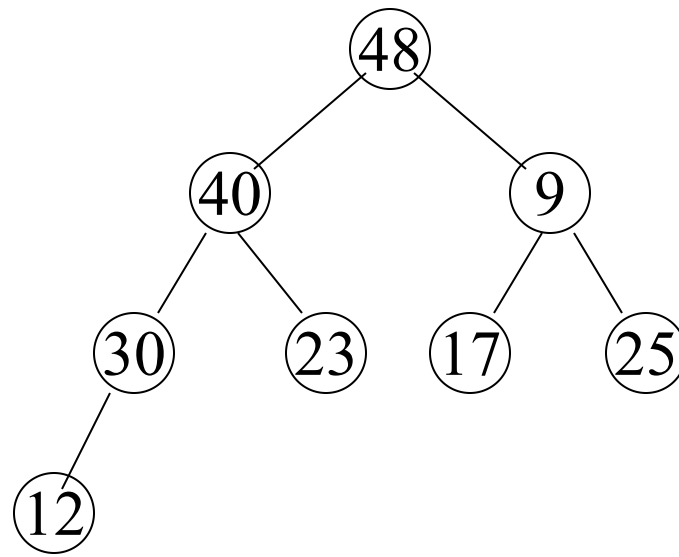


如何把去掉根结点的
二叉树调整成堆？

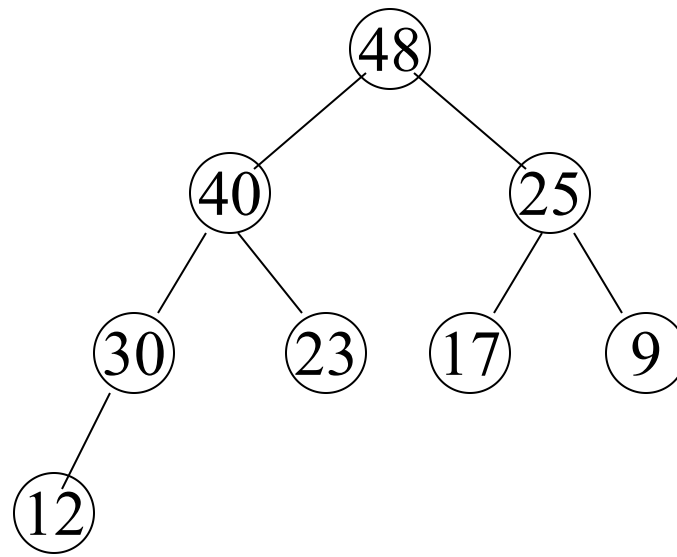




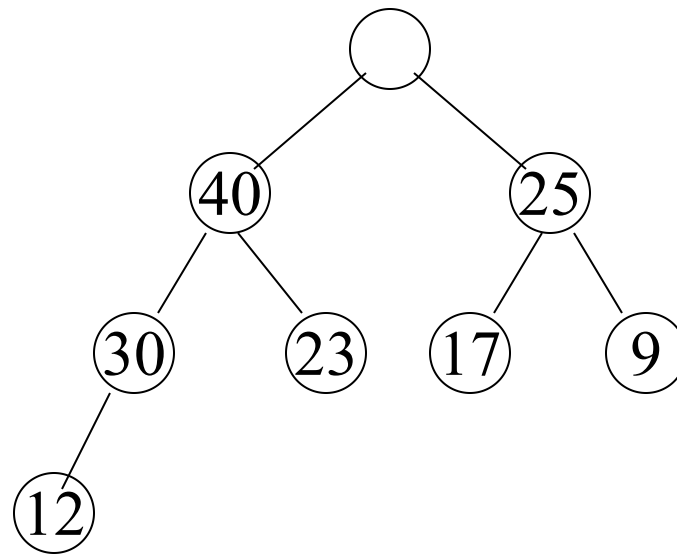
								n
								67



								n
								67



								n
								67

[illegible]

第二个问题的背景:

输出堆顶元素后，将堆底元素送入堆顶（或将堆顶元素与堆底元素交换），堆可能被破坏。

破坏的情况仅是根结点和其左右孩子之间可能不满足堆的特性，而其左右子树仍然是局部的堆。

调整方法:

将根结点与左、右孩子中较大(小顶堆为较小)的进行交换。
若与左孩子交换, 则左子树堆可能被破坏, 且仅左子树的根结点处不满足堆的性质; 若与右孩子交换, 则右子树堆可能被破坏, 且仅右子树的根结点处不满足堆的性质。继续对不满足堆性质的子树进行上述操作, 直到满足了堆性质或者到叶子结点, 堆被建成。

称这个自根结点到叶子结点的调整过程为**筛选**。

[算法描述]

```
typedef SqList  HeapType;
```



```
void HeapAdjust(HeapType &H, int s, int m) ← 子树最后一个结点的序号
```

```
{ rc=H.r[s];
```

```
    j为左孩子的序号
```

```
    for (j←2*s; j≤m; j*=2) {
```

```
        if (j<m && H.r[j].key < H.r[j+1].key)
```

```
            j++; //若左小于右, j++, 即让j指向左右之中最大的那个
```

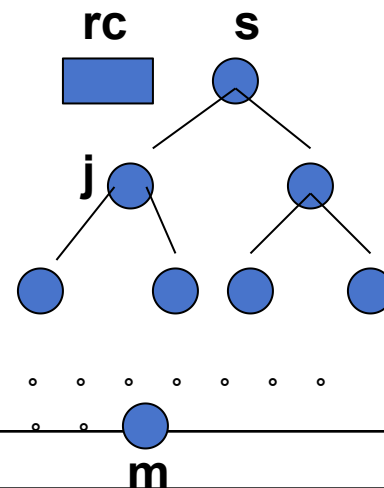
```
        if (rc.key ≥ H.r[j].key) break; //父结点≥孩子中最大的则break
```

```
        H.r[s] = H.r[j]; s=j; //否则交换
```

```
    }
```

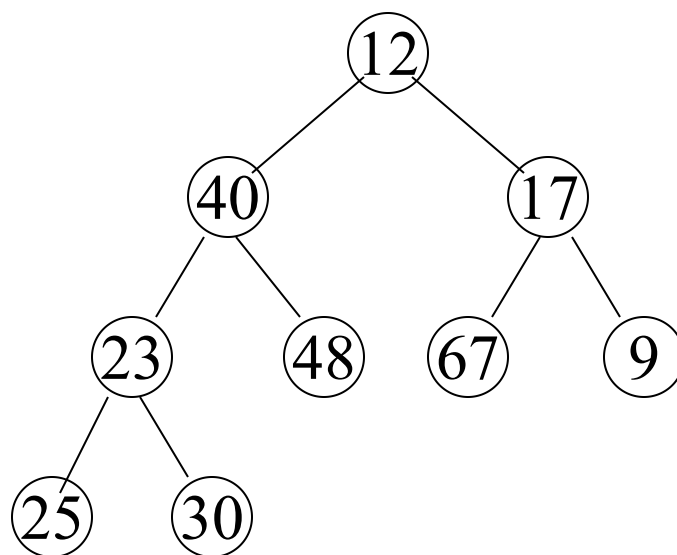
```
    H.r[s]=rc;
```

```
}//HeapAdjust
```



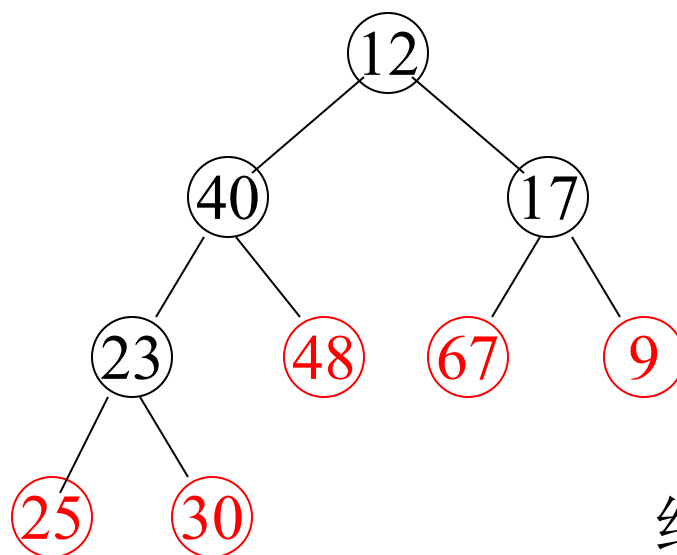
初始化堆

12 40 17 23 48 67 9 25 30



初始化堆

12 40 17 23 48 67 9 25 30



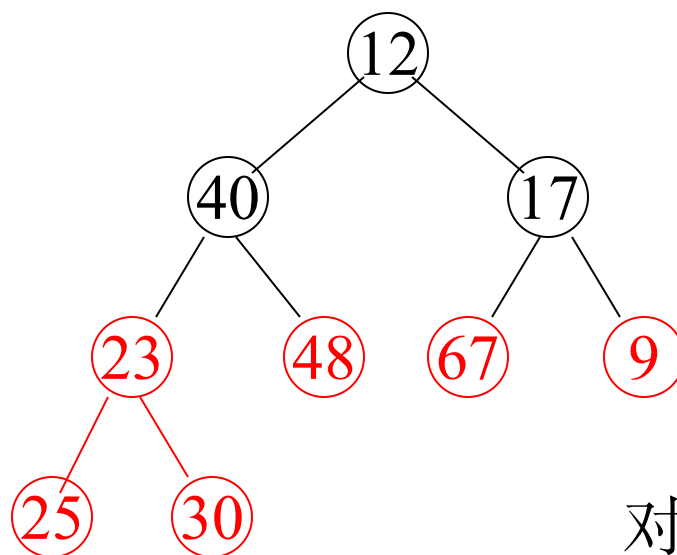
结点 $[n/2]+1, \dots, n$

满足: $a_i \geq a_{2i}$

$a_i \geq a_{2i+1}$

初始化堆

12 40 17 23 48 67 9 25 30

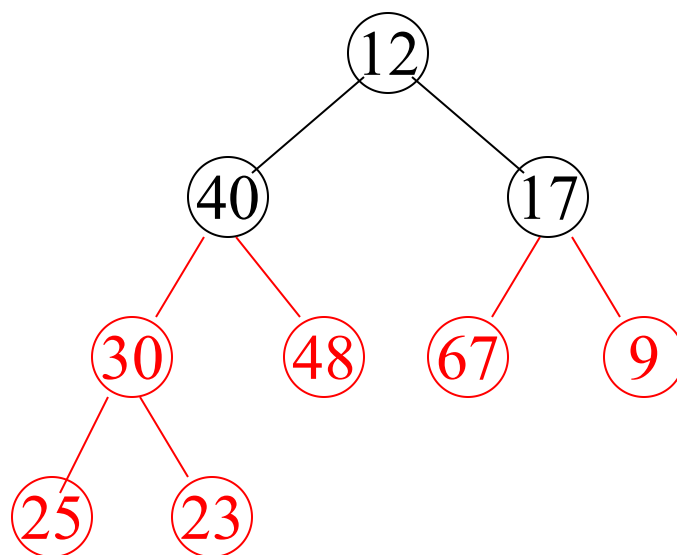


对结点 $[n/2], \dots, n$
进行堆调整:

`heap_just (R,  $[n/2], n$ )`

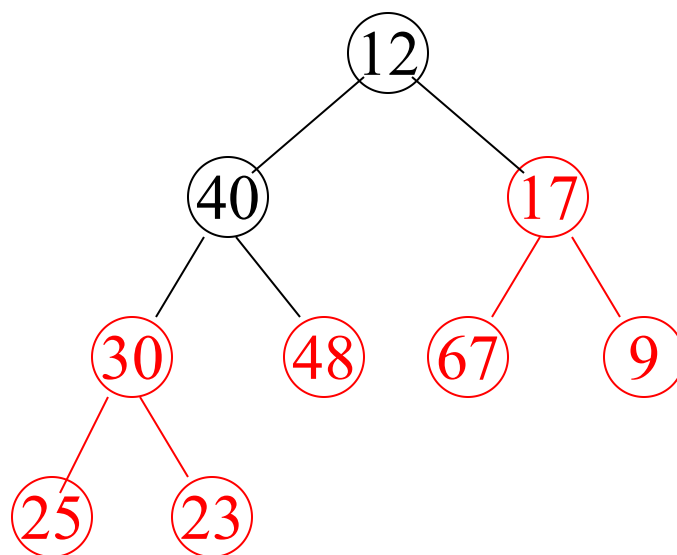
初始化堆

12 40 17 23 48 67 9 25 30



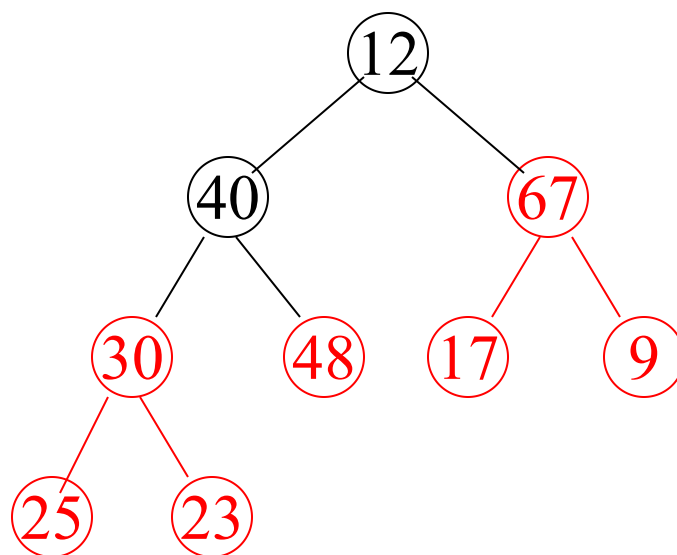
初始化堆

12 40 17 23 48 67 9 25 30



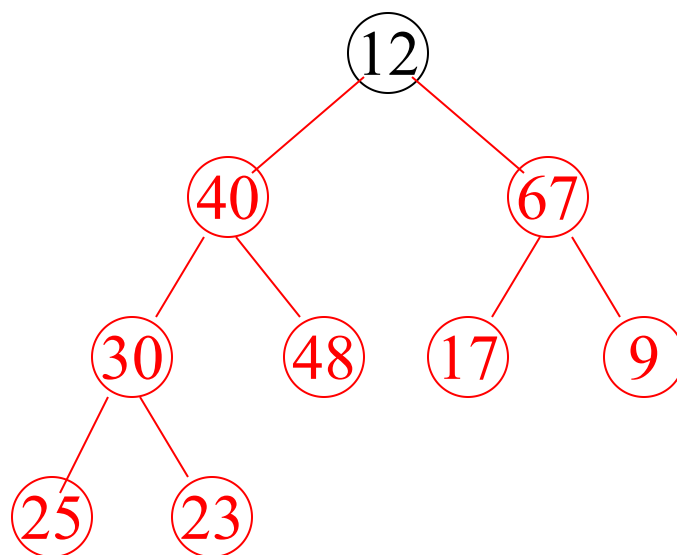
初始化堆

12 40 17 23 48 67 9 25 30



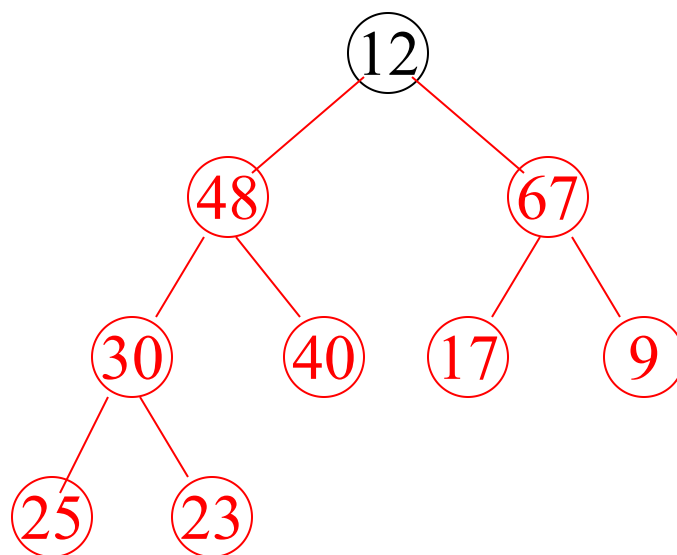
初始化堆

12 40 17 23 48 67 9 25 30



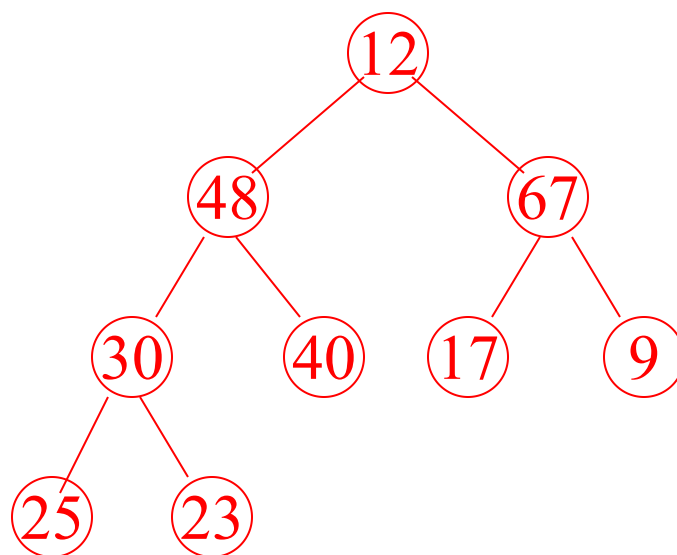
初始化堆

12 40 17 23 48 67 9 25 30



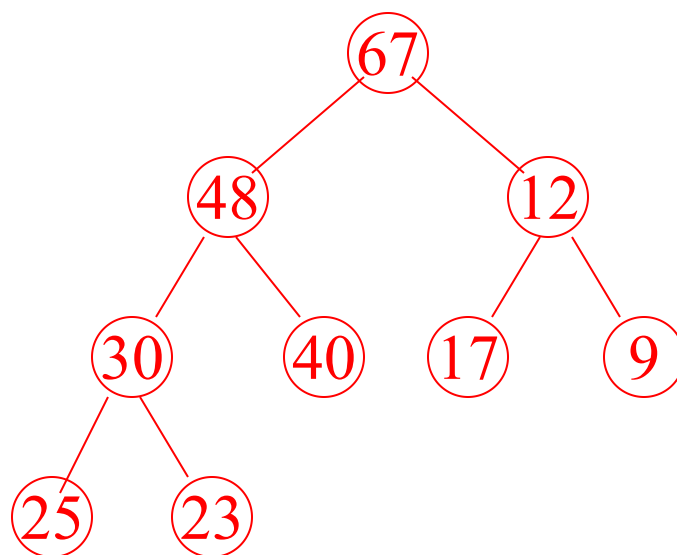
初始化堆

12 40 17 23 48 67 9 25 30



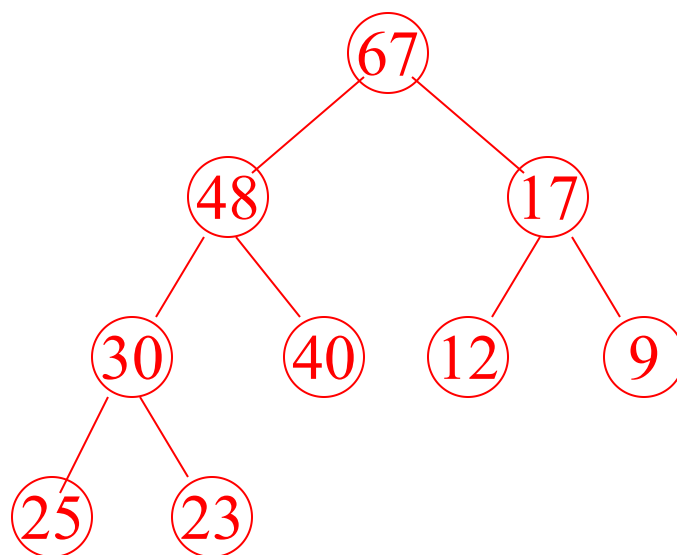
初始化堆

12 40 17 23 48 67 9 25 30



初始化堆

12 40 17 23 48 67 9 25 30



[算法描述]

```
typedef SqList HeapType;
```



```
void HeapSort(HeapType &H)
```

```
{ for (i = H.length / 2; i > 0; i--)
```

```
    HeapAdjust(H, i, H.length); // 初始化堆/
```

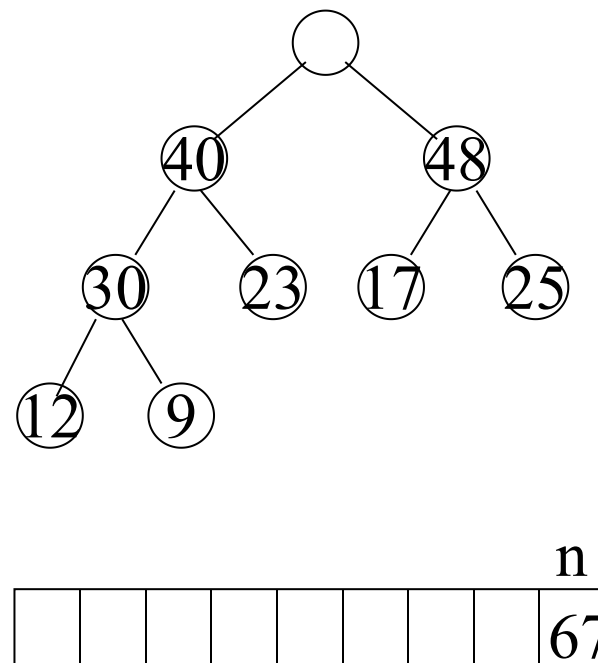
```
    for (i=H.length; i>1; i--) {
```

```
        H.r[1] ←→ H.r[i]; // 将大顶元素拿出/
```

```
        HeapAdjust(H, 1, i-1); // 调整剩余部分
```

```
    }
```

```
} // HeapSort
```



堆排序算法性能:

设树高为 k , 由完全二叉树的性质知

$$k = \lfloor \log_2 n \rfloor + 1$$

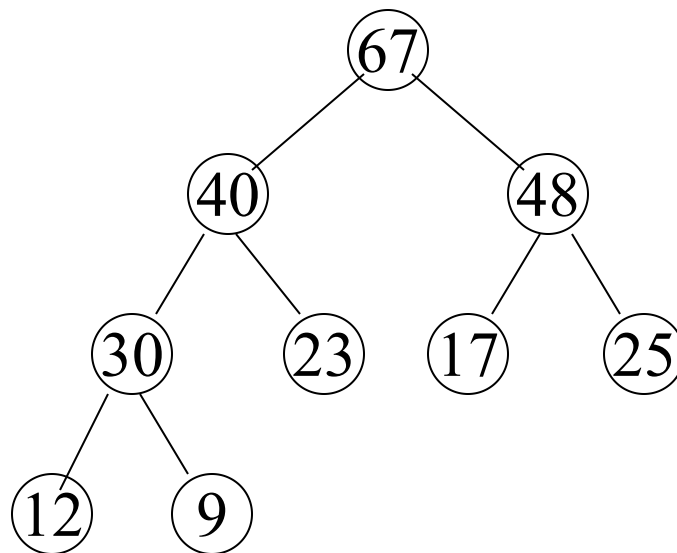
从根到叶子的筛选, 关键码比较次数最多为 $2(k-1)$ 次, 交换记录至多 k 次。

$$T(n) = O(n * \log_2 n)$$

辅助空间复杂度: $O(1)$

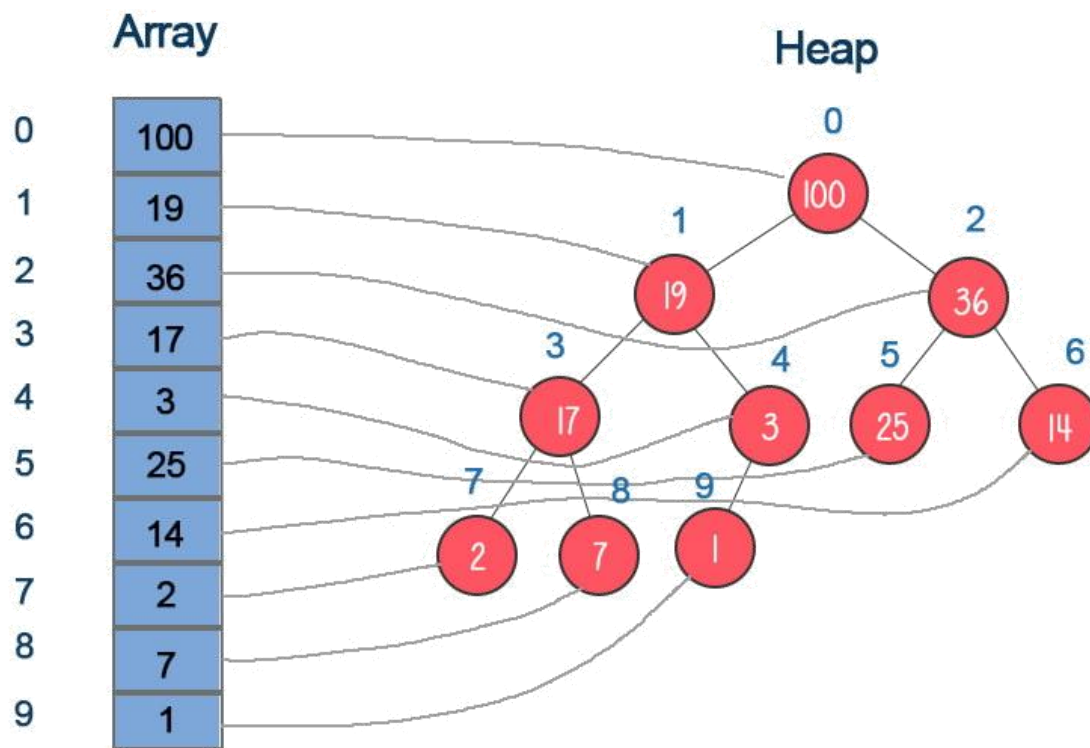
不稳定排序

对记录数较大的文件很有效



优先队列

- 在队列中，执行先进先出规则，而在**优先队列**中，首先出队具有**最高优先级**的元素。
- 二叉堆是对优先队列的一种高效实现。



10.5 归并排序

归并排序的思想是将几个相邻的有序表合并成一个总的有序表，本节主要介绍2-路归并排序。

1. 两个有序表的合并

二路归并排序的基本操作是将两个有序表合并为一个有序表。

[illegible]

R1: 18 25 37 38 46 46 75 78 80
s t

2. 2-路归并算法

2-路归并的基本思想是：

- 只有1个元素的表总是有序的，所以将排序表 $R[1..n]$ ，看作是 n 个长度为 $len=1$ 的有序子表，对相邻的两个有序子表两两合并到 $R[1..n]$ ，使之生成表长 $len=2$ 的有序表；
- 再进行两两合并到 $R[1..n]$ 中， $\cdots$ ，直到最后生成表长 $len=n$ 的有序表。
- 这个过程需要 $\lceil \log_2 n \rceil$ 趟。

初始序列:

56 47 69 48 27 98 56 59 38 28 66



第一趟归并后:

47 56 48 69 27 98 56 59 28 38 66



第二趟归并后:

47 48 56 69 27 56 59 98 28 38 66



第三趟归并后:

27 47 48 56 56 59 69 98 28 38 66



第四趟归并后:

27 28 38 47 48 56 56 59 66 69 98

```
Void Merge(RedType SR[ ], RedType &TR[], int i, int m, int n)
```

```
// 将有序的SR[i..m] 和 SR[m+1..n]归并为有序的TR[i..n]
```

```
{ for (j=m+1, k=i; i<=m && j<=n; k++) {
```

```
    if ( SR[i].key <= SR[j].key) //找到较小者
```

```
        TR[k] = SR[i++]; //将较小者放入TR[k]
```

```
    else
```

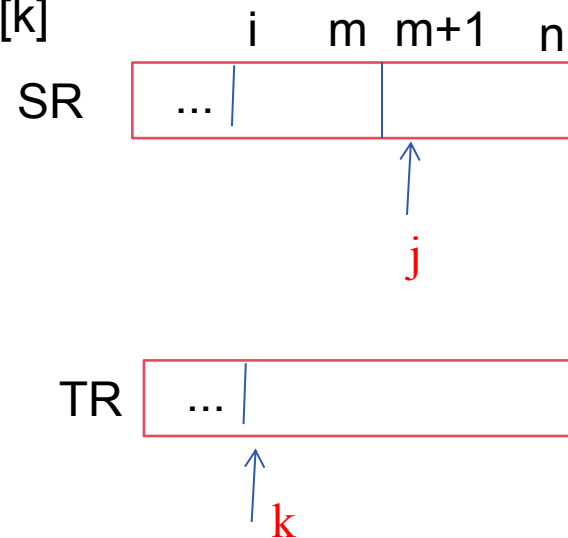
```
        TR[k] = SR[j++];
```

```
}
```

```
if (i<=m) TR[k..n] = SR[i..m]; //处理剩余
```

```
if (j<=n) TR[k..n] = SR[j..n]; //处理剩余
```

```
} //Merge
```



[递归算法描述]

```
void MergeSort(SqList &L)
// 将顺序表 L 进行归并排序
{  Msort ( L.r, L.r, 1, L.length ) ;
} //MergeSort
```

```
void MSort( RedType SR[], RedType &TR1[], int s, int t )
// 将 SR[s..t] 归并排序为 TR1[ s..t]
{  if ( s == t ) TR1[s] = SR[s]; //结束条件
    else {
        m = ( s + t ) / 2 ; // 将SR[s..t]分为SR[s..m]和SR[m+1..t]
        Msort(SR, TR2, s, m); // 将SR[s..m]归并为有序的TR2[s..m]
        Msort(SR, TR2, m+1, t); // 将SR[m+1..t]归并为有序的TR2[m+1..t]
        Merge(TR2, TR1, s, m, t);
        // 将TR2[s..m]和TR2[m+1..t] 归并到TR1[s..t]
    }
} //MSort
```

归并排序性能分析

- 空间性能
 - 需要一个与表等长的辅助元素数据空间，所以其空间复杂度为 $O(n)$
- 时间性能
 - 对 n 个元素的表，可以将 n 个元素看作叶子结点，若将两两归并生成的子表看作它们的父结点，则归并过程对应由叶向根生成一棵二叉树的过程，所以归并趟数约等于二叉树的高度，即 $O(\log_2 n)$ ，每趟归并需移动记录 n 次，故时间复杂度为 $O(n \log_2 n)$
 - 归并排序是一种稳定的排序方法

10.6 分配排序

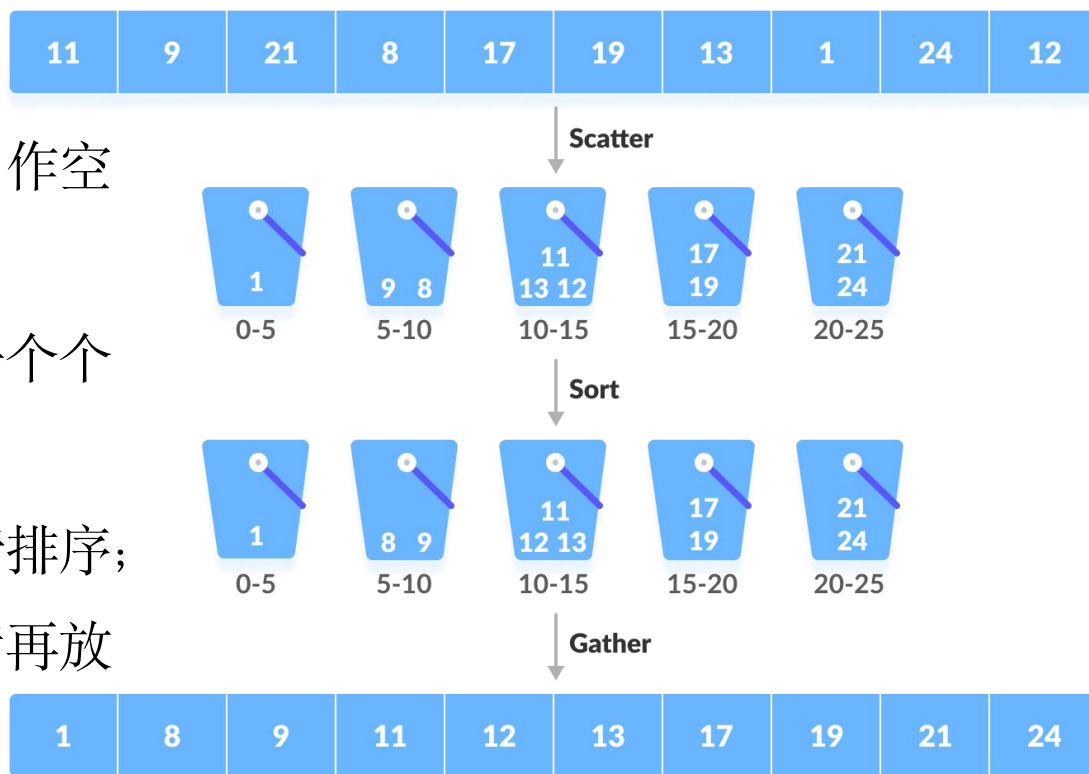
<https://www.cs.usfca.edu/~galles/visualization/BucketSort.html>

通过多次的“分配”和“收集”来完成排序，不需要进行关键字的比较。

10.6.1 桶排序 (Bucket Sort)

将一组数据分割成有限数量的桶（或容器），然后对每个桶中的数据进行了排序，最后将这些桶按顺序合并以得到排好序的数据集。

- 设置一定数量的数组当作空桶；
- 遍历序列，并将元素一个个放到对应的桶中；
- 对每个不是空的桶进行排序；
- 从不是空的桶里把元素再放回原来的序列中。



10.6.1 桶排序 (Bucket Sort)

时间复杂度分析： 分桶 $O(n)$ +收集 $O(n)$ +桶内排序时间复杂度

- 最好情况： 每桶一个元素， 此时时间复杂度为 $O(n)$
- 最坏情况： n 个元素全被分配到一个桶中

稳定性分析： 如果使用稳定的桶内排序， 并且将元素插入桶中时不改变元素间的相对顺序， 那么桶排序就是一种**稳定**的排序算法。

适用情况： 待排序数据值域较大但分布比较均匀。

10.6.2 计数排序 (Counting Sort)

思路：对于给定的输入序列中的每一个元素 x ，确定该序列中值小于 x 的元素个数。一旦有了这个信息，就可以将 x 直接存放到最终的输出序列的正确位置上。例如，如果输入序列中只有17个元素的值小于 x 的值，则 x 可以直接存放在输出序列的第18个位置上。

算法：

- 找出待排序的数组中最大和最小的元素；
- 统计数组中每个值为 i 的元素出现的次数，存入数组 C 的第 i 项；
- 对所有的计数累加（从 C 中的第一个元素开始，每一项和前一项相加）；
- 反向填充目标数组：将每个元素 i 放在新数组的第 $C(i)$ 项，每放一个元素就将 $C(i)$ 减去1。

<https://www.cs.usfca.edu/~galles/visualization/CountingSort.html>

10.6.2 计数排序 (Counting Sort)

array

7	3	5	8	6	7	4	5
---	---	---	---	---	---	---	---

	从索引0开始依次存放3~8出现的次数					
元素	3	4	5	6	7	8
索引	0	1	2	3	4	5
次数	1	1	2	1	2	1

counts

	每个次数累加上其前面的所有次数 得到的就是元素在有序序列中的位置信息					
元素	3	4	5	6	7	8
索引	0	1	2	3	4	5
次数	1	2	4	5	7	8

累加

0	1	2	3	4	5	6	7
3	4	5	5	6	7	7	8

10.6.2 计数排序 (Counting Sort)

复杂度：当输入的元素是 n 个 0 到 k 之间的整数时，时间复杂度是 $O(n+k)$ ，空间复杂度也是 $O(n+k)$ ，其排序速度快于任何比较排序算法。

稳定性：计数排序是一个稳定的排序算法。

适用场景：当 k 不是很大并且序列比较集中时，计数排序是一个很有
效的排序算法。当 k 大于 n 时，效率下降。

10.6.3 基数排序

基数排序是一种借助于**多关键码排序**的思想，是将单关键码按基数分成“多关键码”进行排序的方法。

扑克牌中52张牌，可按**花色**和**面值**分成两个字段，其大小关系为：

- 花色：♣ < ♦ < ♥ < ♠
- 面值：2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A

若对扑克牌按花色、面值进行升序排序（花色优先），得到如下序列：

♣2,3,...,A, ♦2,3,...,A, ♥2,3,...,A, ♠2,3,...,A

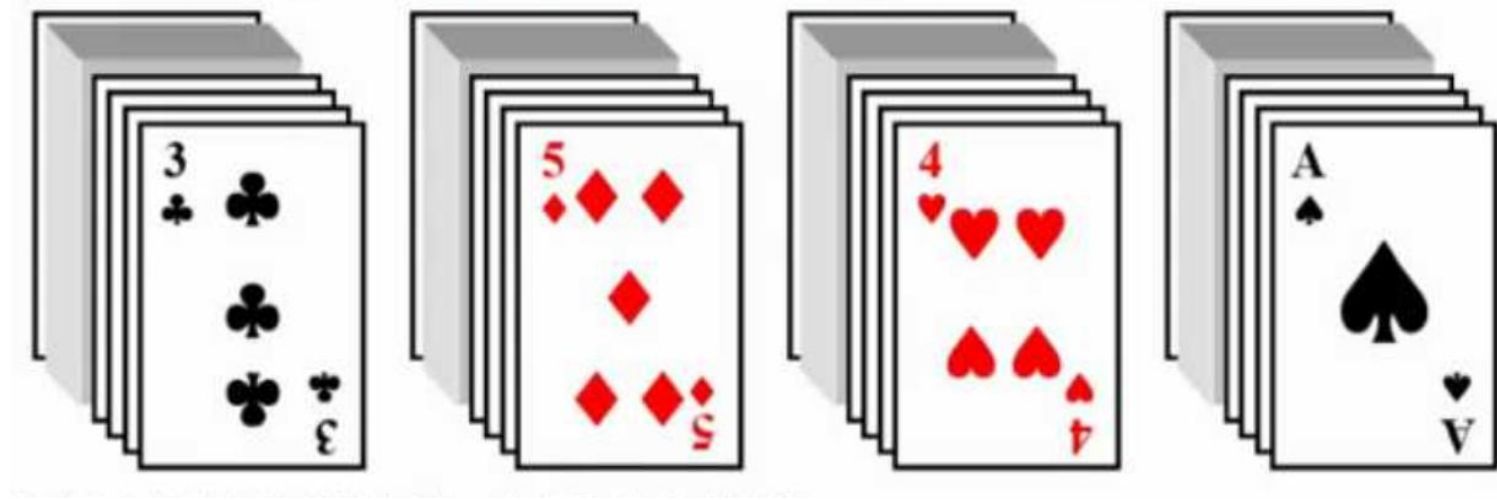
这就是多关键码排序。

<https://www.cs.usfca.edu/~galles/visualization/RadixSort.html>

多关键码排序按照从最主位关键码到最次位关键码或从最次位到最主位关键码的顺序逐次排序。

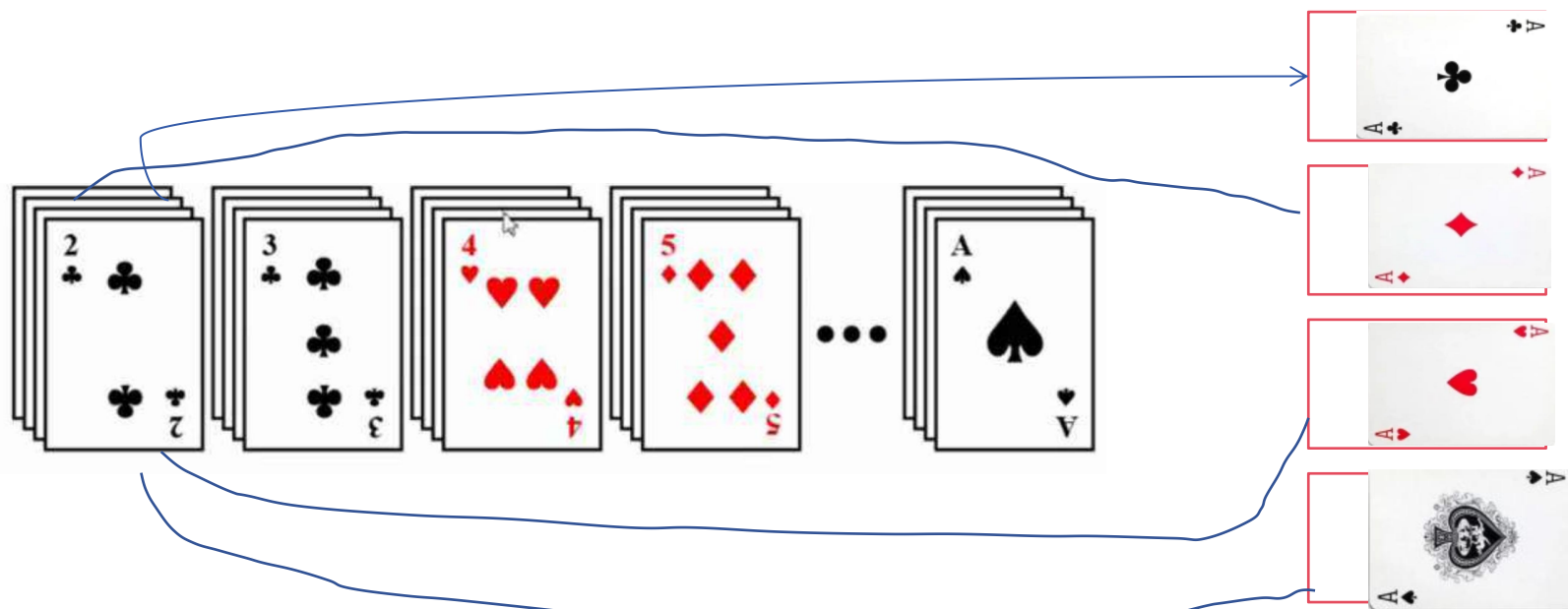
方法1: 最高位优先(Most Significant Digit first)法, 简称MSD法

先对花色排序, 将其分为4个组, 即梅花组、方块组、红心组、黑心组。
再对每个组分别按面值进行排序, 最后, 将4个组连接起来即可。



方法2: 最低位优先(Least Significant Digit first)法, 简称LSD法.

先按13个面值给出13个编号组(2号, 3号, ..., A号), 将牌按面值依次放入对应的编号组, 分成13堆。再按花色给出4个编号组(梅花、方块、红心、黑心), 将2号组中牌取出分别放入对应花色组, 再将3号组中牌取出分别放入对应花色组, ……., 这样, 4个花色组中均按面值有序, 然后, 将4个花色组依次连接起来即可。



从下至上依次为2、3、……、K、A

3

44

38

5

47

15

36

26

27

2

46

4

19

50

48

[基数排序算法思想] 借鉴LSD法

设参加排序的序列关键字为 $K=\{K_1, K_2, \dots, K_n\}$,

其中 K_i 是 d 位 rd 进制的数, rd 称为**基数**;

d 由所有元素中最长的一个元素的位数计量,

$$K_i = K_i^0 K_i^1 \dots K_i^{d-1}$$

高 ← 低

从低位到高位依次对 $K^j(j=d-1, d-2, \dots, 0)$ 根据**基数分配**, 再按**基数递增序收集**, 则可得有序序列。

[例]

(1) $K=\{3621 \text{ } 0724 \text{ } 8385 \text{ } 0075 \text{ } 0514 \text{ } 7368 \text{ } 0008 \}$

$rd=10, d=4$

(2) $K=\{\text{Zhang Wang Li} \text{DEL DEL DEL} \text{ Zhao} \text{DEL}\}$

$rd=27, d=5$

基于计数的基数排序

475	572	311	798	821	606	823	597	816	768	471	839	879	546	716	825	748	53	616	973	867	25	384	848	331	31	719	501	171	613
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----	-----

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29

--	--	--	--	--	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29

<https://www.cs.usfca.edu/~galles/visualization/RadixSort.html>

基于计数的基数排序

475	572	311	798	821	606	823	597	816	768	471	839	879	546	716	825	748	53	616	973	867	25	384	848	331	31	719	501	171	613
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29

0	1	1	0	0	1	0	0	1	0
0	1	2	3	4	5	6	7	8	9

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	

<https://www.cs.usfca.edu/~galles/visualization/RadixSort.html>

基于计数的基数排序

475	572	311	798	821	606	823	597	816	768	471	839	879	546	716	825	748	53	616	973	867	25	384	848	331	31	719	501	171	613
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29

0	7	1	4	1	3	5	2	4	3
0	1	2	3	4	5	6	7	8	9

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	

完成按个位的计数

<https://www.cs.usfca.edu/~galles/visualization/RadixSort.html>

基于计数的基数排序

475	572	311	798	821	606	823	597	816	768	471	839	879	546	716	825	748	53	616	973	867	25	384	848	331	31	719	501	171	613
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29

0	7	8	12	13	16	21	23	27	30
0	1	2	3	4	5	6	7	8	9

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29

对个位计数累加

<https://www.cs.usfca.edu/~galles/visualization/RadixSort.html>

基于计数的基数排序

475	572	311	798	821	606	823	597	816	768	471	839	879	546	716	825	748	53	616	973	867	25	384	848	331	31	719	501	171	613
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29

0	7	8	11	13	16	21	23	27	30
0	1	2	3	4	5	6	7	8	9

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	

个位为“3”的放置在下标为11开始的格子

<https://www.cs.usfca.edu/~galles/visualization/RadixSort.html>

基于计数的基数排序

475	572	311	798	821	606	823	597	816	768	471	839	879	546	716	825	748	53	616	973	867	25	384	848	331	31	719	501	171	613
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29

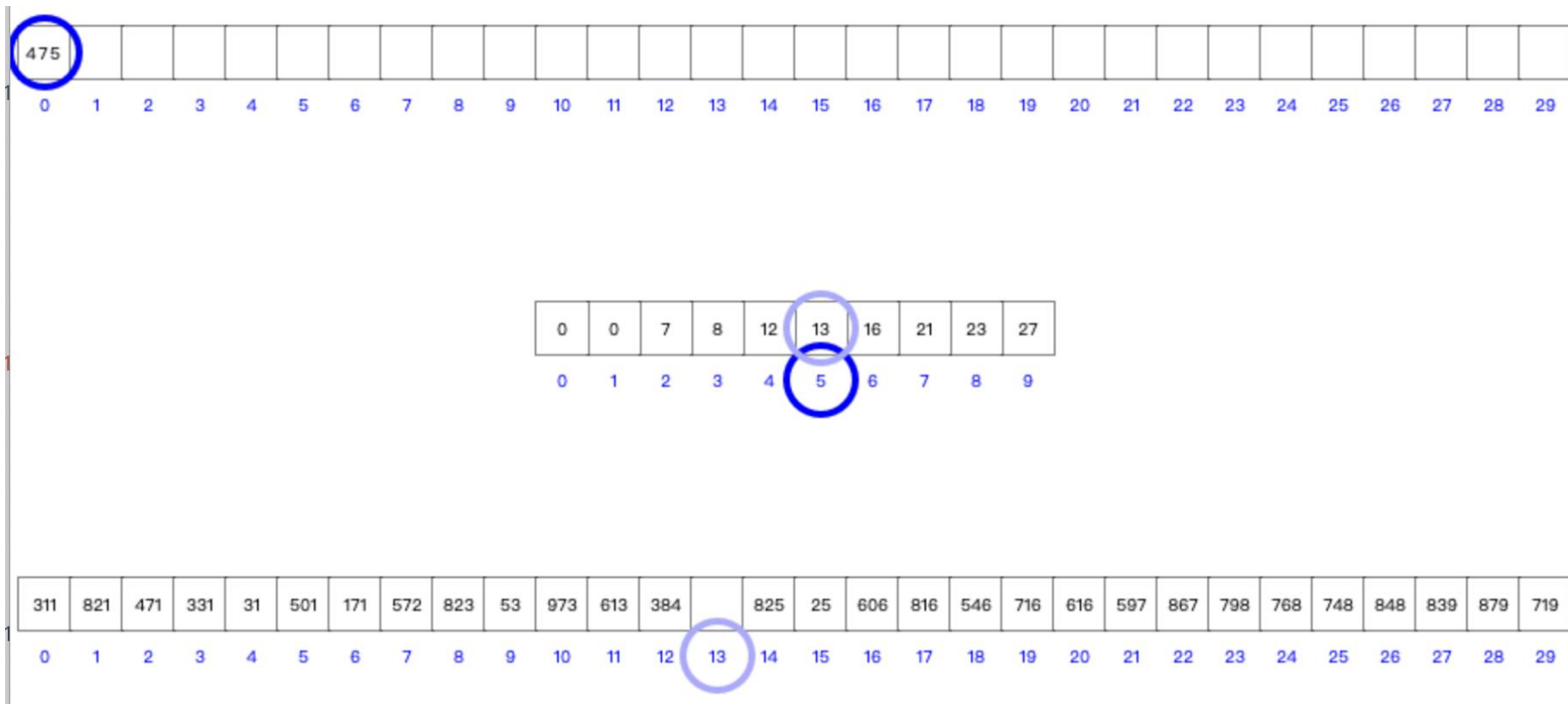
0	7	8	11	13	16	21	23	27	30
0	1	2	3	4	5	6	7	8	9

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	

个位为“3”的放置在下标为11开始的格子

<https://www.cs.usfca.edu/~galles/visualization/RadixSort.html>

基于计数的基数排序



按个位排序完毕

<https://www.cs.usfca.edu/~galles/visualization/RadixSort.html>

[链式基数排序示例] { 477 241 467 5 363 81 5 }

rd=10
d=3

第1趟: 分配 [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

按个位
 ↓241 ↓363 ↓5 ↓477
 ↓81 ↓5 ↓467

收集 {241 81 363 5 5 477 467}

第2趟: 分配 [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

按十位
 ↓5 241 ↓363 477 81
 ↓5 ↓467

收集 {5 5 241 363 467 477 81}

第3趟: 分配 [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

按百位
 ↓5 241 363 ↓467
 ↓5 ↓477
 ↓81

收集 {5 5 81 241 363 467 477}

稳定排序

链队的个数=rd=10

[链式基数排序示例]

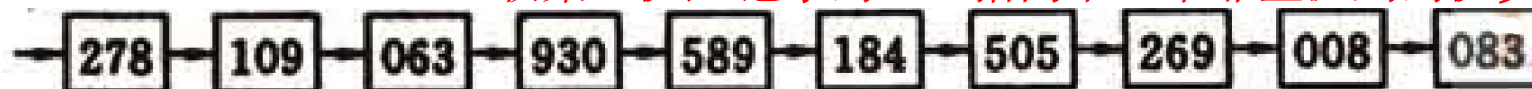
rd=10

d=3

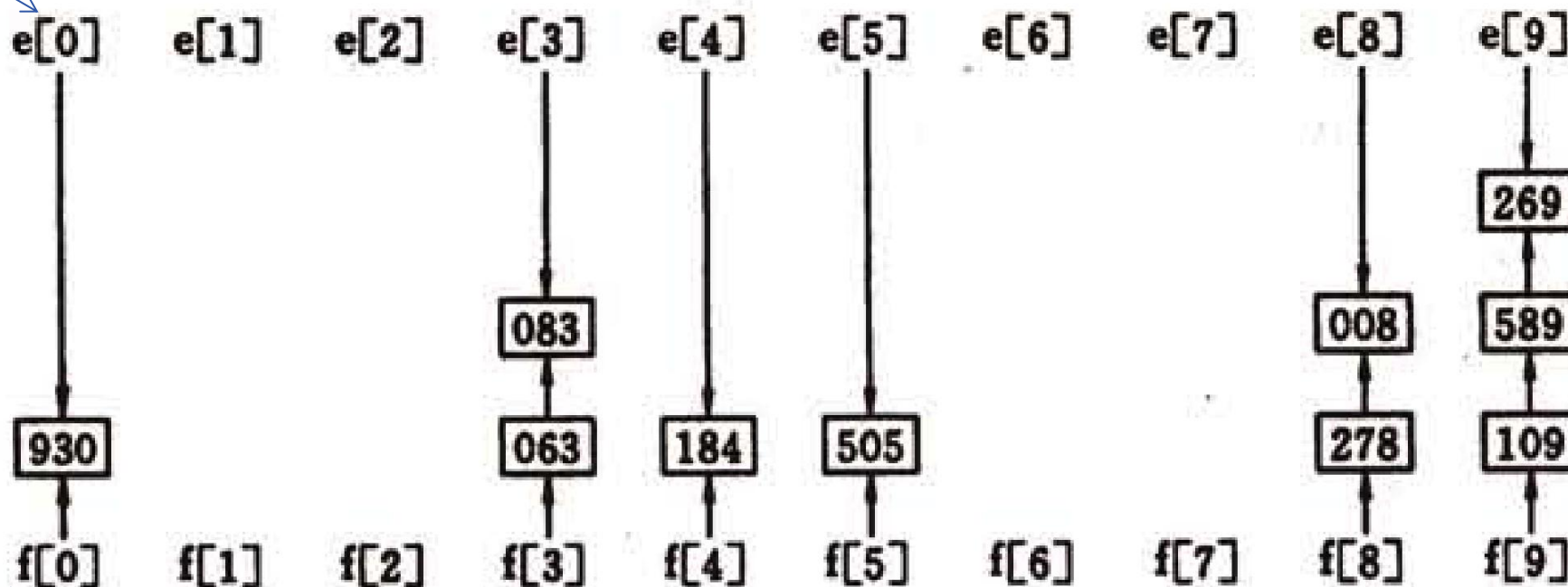
分配：改变记录的指针，进入对应的链队，

收集：队尾记录的next指向下一个非空队列的队头记录

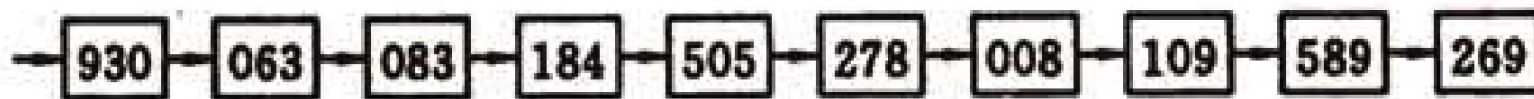
主静态链表



链队尾指针



链队头指针



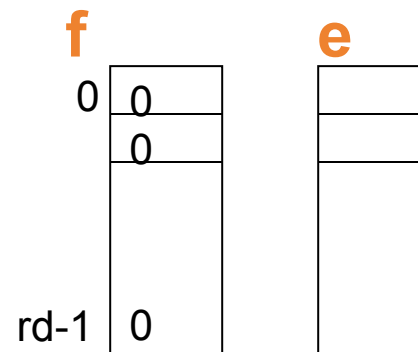
[数据结构]

- 主：静态链表 $r[0..n]$
 - 数据域($\text{key} + \text{otheritem}$)：记录
 - 指针域(next)：下一个记录的序号
- 辅： $f[0..rd-1]$ 各链式队列头指针
 $e[0..rd-1]$ 各链式队列尾指针

$r[0..n]$

	key	otheritem	next
0			1
1			2
2			3
n			0

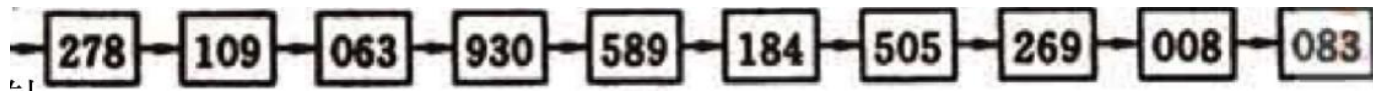
[数据结构定义]



[数据结构]

```
#define MAX_NUM_OF_KEY 8 //可容纳的最多子关键字个数
#define RADIX 10 //关键字的基数rd
#define MAX_SPACE 10000 //记录空间可容纳的最多记录数
typedef struct {
    KeysType key[MAX_NUM_OF_KEY];
    InfoType otheritem;
    int next;
}SLCell;
typedef struct {
    SLCCell r[MAX_SPACE]; //r[0]为头结点
    int keynum; //实际划分的关键字位数d
    int recnum; //实际记录数
}SLList;
Typedef int ArrType[RADIX]; //ArrType为数组类型
```

[算法描述]



主过程 RadixSort

分配过程: Distribute

收集过程: Collect

下标	关键字	next
0		1
1	278	2
2	109	3
3	063	4
4	930	5
5	589	6
6	184	7
7	505	8
8	269	9
9	008	10
10	083	0

```
void RadixSort (SLList &L)
```

```
{  for (i = 0; i<L.recnum; i++)
```

```
    L.r [ i ].next = i+1;
```

```
    L.r[L.recnum].next = 0;  //将L改造为静态链表
```

```
    for (i = 0; i<L.keynum; i++) { //LSD,从最低位开始
```

```
        Distribute(L.r, i, f, e); //分配
```

```
        Collect(L.r, i, f, e);  //收集
```

```
    }
```

```
} //RadixSort
```

```

void Distribute (SLCell &r, int i, ArrType &f, ArrType &e)
{
    for (j=0; j<RADIX; j++)    f[j]=0; //链队队头指向0
    for (p=r[0].next; p; p=r[p].next) { //依次处理每个记录
        j=ord(r[p].keys[i]); //求记录中关键字的第i位的队列编号
        if (!f[j]) f[j]=p; //如果原来链队为空
        else r[e[j]].next=p; //原来链队不为空
        e[j]=p; //链队尾指针指向新加入的记录
    }
} //Distribute

```

下标	关键字	next
0		1
1	278	2
2	109	3
3	063	4
4	930	5
5	589	6
6	184	7
7	505	8
8	269	9
9	008	10
10	083	0

队列编号	静态指针
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

f: 链队队头

队列编号	静态指针
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

e: 链队队尾

```

void Distribute (SLCell &r, int i, ArrType &f, ArrType &e)
{
    for (j=0; j<RADIX; j++)    f[j]=0;  //链队队头指向0
    for (p=r[0].next; p; p=r[p].next) { //依次处理每个记录
        j=ord(r[p].keys[i]); //求记录中关键字的第i位的队列编号
        if (!f[j]) f[j]=p; //如果原来链队为空
        else r[e[j]].next=p; //原来链队不为空
        e[j]=p; //链队尾指针指向新加入的记录
    }
} //Distribute

```

下标	关键字	next
0		1
1	278	2
2	109	3
3	063	4
4	930	5
5	589	6
6	184	7
7	505	8
8	269	9
9	008	10
10	083	0

队列编号	静态指针
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	1
9	0

f: 链队队头

队列编号	静态指针
0	
1	
2	
3	
4	
5	
6	
7	
8	1
9	

e: 链队队尾

```

void Distribute (SLCell &r, int i, ArrType &f, ArrType &e)
{
    for (j=0; j<RADIX; j++)    f[j]=0;  //链队队头指向0
    for (p=r[0].next; p; p=r[p].next) { //依次处理每个记录
        j=ord(r[p].keys[i]); //求记录中关键字的第i位的队列编号
        if (!f[j]) f[j]=p; //如果原来链队为空
        else r[e[j]].next=p; //原来链队不为空
        e[j]=p; //链队尾指针指向新加入的记录
    }
} //Distribute

```

下标	关键字	next
0		1
1	278	2
2	109	3
3	063	4
4	930	5
5	589	6
6	184	7
7	505	8
8	269	9
9	008	10
10	083	0

队列编号	静态指针
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	1
9	2

f: 链队队头

队列编号	静态指针
0	
1	
2	
3	
4	
5	
6	
7	
8	1
9	2

e: 链队队尾


```

void Distribute (SLCell &r, int i, ArrType &f, ArrType &e)
{
    for (j=0; j<RADIX; j++)    f[j]=0;    //链队队头指向0
    for (p=r[0].next; p;    p=r[p].next) { //依次处理每个记录
        j=ord(r[p].keys[i]); //求记录中关键字的第i位的队列编号
        if (!f[j]) f[j]=p; //如果原来链队为空
        else r[e[j]].next=p; //原来链队不为空
        e[j]=p; //链队尾指针指向新加入的记录
    }
} //Distribute

```

下标	关键字	next
0		1
1	278	2
2	109	3
3	063	4
4	930	5
5	589	6
6	184	7
7	505	8
8	269	9
9	008	10
10	083	0

队列编号	静态指针
0	0
1	0
2	0
3	3
4	0
5	0
6	0
7	0
8	1
9	2

队列编号	静态指针
0	
1	
2	
3	3
4	
5	
6	
7	
8	1
9	2

f: 链队队头

e: 链队队尾

```

void Distribute (SLCell &r, int i, ArrType &f, ArrType &e)
{
    for (j=0; j<RADIX; j++)    f[j]=0;  //链队队头指向0
    for (p=r[0].next; p;    p=r[p].next) { //依次处理每个记录
        j=ord(r[p].keys[i]); //求记录中关键字的第i位的队列编号
        if (!f[j]) f[j]=p; //如果原来链队为空
        else r[e[j]].next=p; //原来链队不为空
        e[j]=p; //链队尾指针指向新加入的记录
    }
} //Distribute

```

下标	关键字	next
0		1
1	278	2
2	109	5
3	063	4
4	930	5
5	589	6
6	184	7
7	505	8
8	269	9
9	008	10
10	083	0

队列编号	静态指针
0	4
1	0
2	0
3	3
4	0
5	0
6	0
7	0
8	1
9	2

f: 链队队头

队列编号	静态指针
0	4
1	
2	
3	3
4	
5	
6	
7	
8	1
9	2->5

e: 链队队尾

```

void Distribute (SLCell &r, int i, ArrType &f, ArrType &e)
{
    for (j=0; j<RADIX; j++)    f[j]=0;  //链队队头指向0
    for (p=r[0].next; p;    p=r[p].next) { //依次处理每个记录
        j=ord(r[p].keys[i]); //求记录中关键字的第i位的队列编号
        if (!f[j]) f[j]=p; //如果原来链队为空
        else r[e[j]].next=p; //原来链队不为空
        e[j]=p; //链队尾指针指向新加入的记录
    }
} //Distribute

```

下标	关键字	next
0		1
1	278	2
2	109	5
3	063	4
4	930	5
5	589	8
6	184	7
7	505	8
8	269	9
9	008	10
10	083	0

队列编号	静态指针
0	4
1	0
2	0
3	3
4	6
5	7
6	0
7	0
8	1
9	2

f: 链队队头

队列编号	静态指针
0	4
1	
2	
3	3
4	6
5	7
6	
7	
8	1
9	5->8

e: 链队队尾

```

void Distribute (SLCell &r, int i, ArrType &f, ArrType &e)
{
    for (j=0; j<RADIX; j++)    f[j]=0;  //链队队头指向0
    for (p=r[0].next; p;  p=r[p].next) { //依次处理每个记录
        j=ord(r[p].keys[i]); //求记录中关键字的第i位的队列编号
        if (!f[j]) f[j]=p; //如果原来链队为空
        else r[e[j]].next=p; //原来链队不为空
        e[j]=p; //链队尾指针指向新加入的记录
    }
} //Distribute

```

下标	关键字	next
0		1
1	278	9
2	109	5
3	063	10
4	930	5
5	589	8
6	184	7
7	505	8
8	269	9
9	008	10
10	083	0

队列编号	静态指针
0	4
1	0
2	0
3	3
4	6
5	7
6	0
7	0
8	1
9	2

f: 链队队头

队列编号	静态指针
0	4
1	
2	
3	10
4	6
5	7
6	
7	
8	9
9	8

e: 链队队尾

```
void Collect (SLCell &r, int i, ArrType &f, ArrType &e)
```

```
{ for (j=0; !f[j]; j++); //找第一个非空子队列
```

```
    r[0].next=f[j]; t=e[j];
```

```
    while (j<RADIX) {
```

```
        for (j++; j<RADIX-1 && !f[j]; j++); //找下一个非空子队列
```

```
        if ( f[j] ) { r[t].next=f[j]; t=e[j];} //链接
```

```
    }
```

```
    r[t].next=0;
```

```
}//Collect
```

下标	关键字	next
0		4
1	278	9
2	109	5
3	063	10
4	930	5
5	589	8
6	184	7
7	505	8
8	269	9
9	008	10
10	083	0

队列编号	静态指针
0	4
1	0
2	0
3	3
4	6
5	7
6	0
7	0
8	1
9	2

f: 链队队头

队列编号	静态指针
0	4
1	
2	
3	10
4	6
5	7
6	
7	
8	9
9	8

e: 链队队尾

```

void Collect (SLCell &r, int i, ArrType &f, ArrType &e)
{
    for (j=0; !f[j]; j++); //找第一个非空子队列
        r[0].next=f[j]; t=e[j];
    while (j<RADIX) {
        for (j++; j<RADIX-1 && !f[j]; j++); //找下一个非空子队列
        if ( f[j] ) { r[t].next=f[j]; t=e[j];} //链接
    }
    r[t].next=0;
} //Collect

```

下标	关键字	next
0		4
1	278	9
2	109	5
3	063	10
4	930	3
5	589	8
6	184	7
7	505	8
8	269	9
9	008	10
10	083	6

队列编号	静态指针
0	4
1	0
2	0
3	3
4	6
5	7
6	0
7	0
8	1
9	2

f: 链队队头

队列编号	静态指针
0	4
1	
2	
3	10
4	6
5	7
6	
7	
8	9
9	8

e: 链队队尾

```
void Collect (SLCell &r, int i, ArrType &f, ArrType &e)
```

```
{ for (j=0; !f[j]; j++); //找第一个非空子队列
```

```
    r[0].next=f[j]; t=e[j];
```

```
    while (j<RADIX) {
```

```
        for (j++; j<RADIX-1 && !f[j]; j++); //找下一个非空子队列
```

```
        if ( f[j] ) { r[t].next=f[j]; t=e[j];} //链接
```

```
    }
```

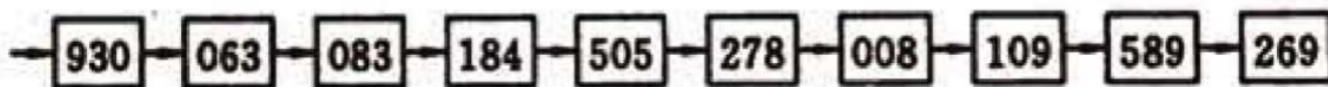
```
    r[t].next=0;
```

```
}//Collect
```

下标	关键字	next
0		4
1	278	9
2	109	5
3	063	10
4	930	3
5	589	8
6	184	7
7	505	1
8	269	0
9	008	2
10	083	6

队列编号	静态指针
0	4
1	0
2	0
3	3
4	6
5	7
6	0
7	0
8	1
9	2

队列编号	静态指针
0	4
1	
2	
3	10
4	6
5	7
6	
7	
8	9
9	8



e: 链队队尾

[链式基数排序的性能分析]

设记录数为 n , 关键字位数为 d , 基数为 rd

每一趟: 分配 $O(n)$ 收集 $O(rd)$

d 趟总计: $O(d(n+rd)) \Rightarrow O(n)$ 通常 d, rd 均为常数

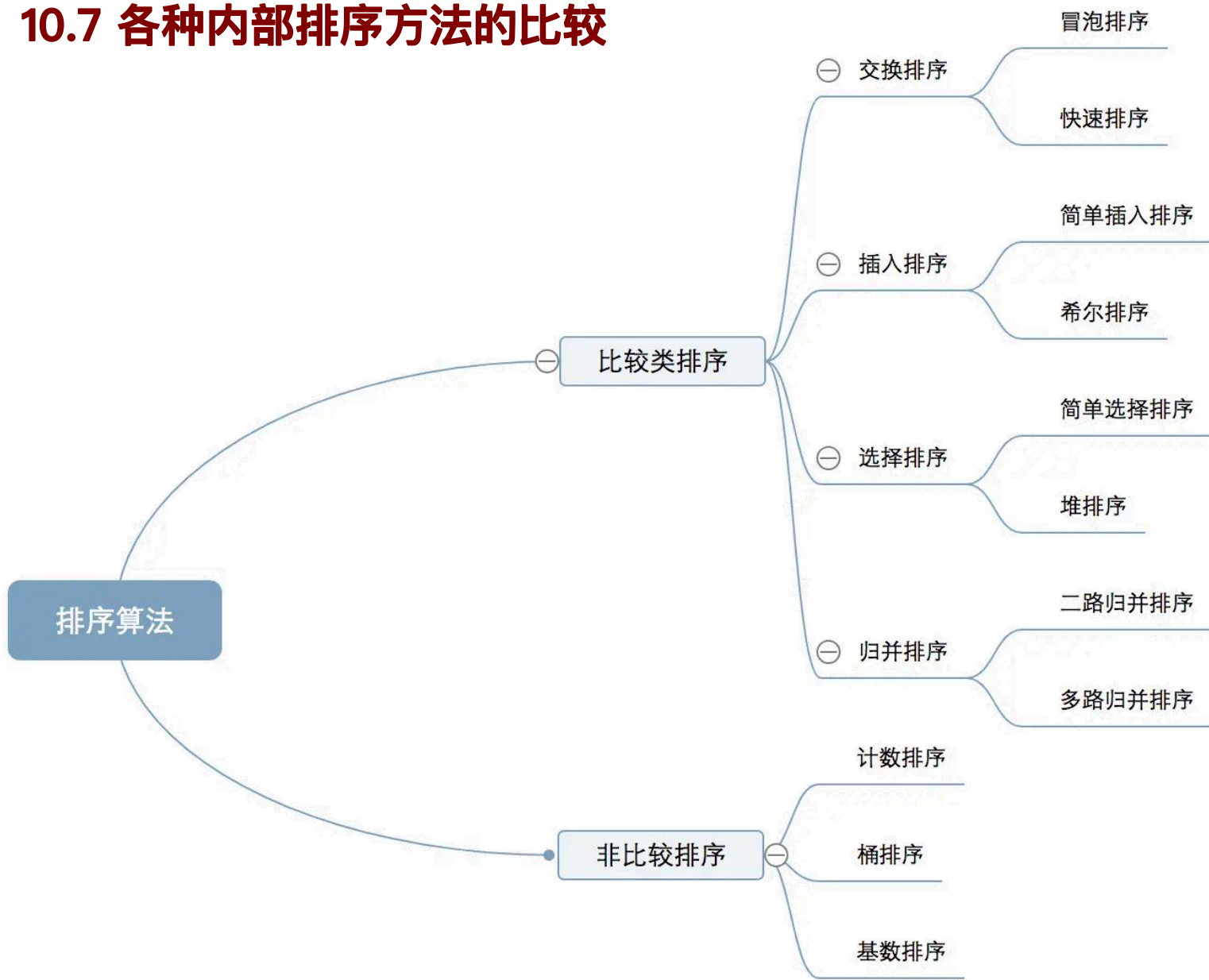
辅助空间

n 个指针域 (游标) 空间

队头指针数组 $f[0..rd-1]$ 和队尾指针数组 $e[0..rd-1]$

辅助空间复杂度: $O(rd+n)$

10.7 各种内部排序方法的比较



10.7 各种内部排序方法的比较

排序方法	最好时间	平均时间	最坏时间	辅助空间	稳定性
简单插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	√
希尔排序	$O(n)$	$O(n^{1.3})$	$O(n^2)$	$O(1)$	×
冒泡 排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	√
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	×
快速排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	×
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	×
归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	√
桶排序	$O(n)$		$O(n^2)$	$O(n+k)$	√
计数排序	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	√
基数排序	$O(d(rd+n))$	$O(d(rd+n))$	$O(d(rd+n))$	$O(rd+n)$	√

- 按平均时间排序方法分为四类
 $O(n^2)$ 、 $O(n\log n)$ 、 $O(n^{1+\varepsilon})$ 、 $O(n)$
- 快速排序是目前**基于比较的**内部排序中最好的方法
- 关键字**随机分布**时，**快速排序的平均时间最短**，堆排序次之，但后者所需的辅助空间少
- 当**n较小**时如($n < 50$)，可采用直接插入或简单选择排序，前者是稳定排序，但后者通常记录移动次数少于前者
- 当**n较大**时，应采用时间复杂度为 $O(n\log n)$ 的排序方法(主要为快速排序和堆排序)或者基数排序的方法，但基数排序对关键字的结构有一定要求
- 当**n较大**时，为避免顺序存储时大量移动记录的时间开销，可考虑用链表作为存储结构（如插入排序、归并排序、基数排序）

- 堆排序难于在链表上实现，可以采用地址排序的方法，之后再按辅助表的次序重排各记录
- 文件初态基本按正序排列时，应选用直接插入、冒泡或随机的快速排序
- 选择排序方法应综合考虑各种因素

讨论：假设有 n 个值不同的元素存于顺序结构中，要求不经排序选出前 k ($k \ll n$) 个最小元素，问哪些方法可用，哪些方法比较次数最少？这 k 个元素也要有序如何？

- 选择排序或冒泡排序： k 趟（数据比较次数约为 kn 次）；
- 快速排序：每次仅对第一个子序列划分，直至子序列长度小于等于 k ；长度不足 k ，则再对其后的子序列划分出补足的长度即可（平均对数据比较 $2n$ 次）
- 堆排序：先建小根堆， $k-1$ 次堆调整（数据比较次数约为 $4n + (k-1)\log_2 n$ ）